# Índice general

**1. Marco Teórico y Estado del Arte** **2**
1.1. El Paradigma de la Accountability y el Marco Regulatorio . . . . 2
1.2. Inteligencia Artificial Conexionista (Deep Learning) . . . . . 6
1.3. Inteligencia Artificial Simbólica y Sistemas Basados en Reglas . . 10
1.4. Inteligencia Artificial Neuro-Simbólica . . . . . 13

**2. Definición del Dominio: Originación de Crédito Bancario** **17**
2.1. Justificación del Escenario . . . . . 17
2.2. Marco Regulatorio Específico del Sector Financiero . . . . . 19
2.3. Modelado del Dominio y Ontología Financiera . . . . . 21
2.4. Catálogo de Reglas en la Concesión de Crédito . . . . . 24
2.5. Perfiles de Usuario . . . . . 27
2.6. Requisitos de Diseño Derivados del Dominio . . . . . 29

**3. Implementación de la arquitectura propuesta** **32**
3.1. Visión general de la arquitectura . . . . . 32
3.2. Decisiones tecnológicas . . . . . 35
3.3. Arquitectura e infraestructura del sistema . . . . . 37
3.4. El *Information State Update* (ISU) . . . . . 40
3.5. Orquestación del flujo conversacional . . . . . 42
3.6. Componente neuronal: Mistral . . . . . 43
3.7. Componente simbólico (Drools) . . . . . 45
3.8. Implementación de las reglas de negocio . . . . . 47
3.9. Entorno bancario simulado . . . . . 50
3.10. Persistencia y trazabilidad . . . . . 51
3.11. Interfaz de usuario y consola de auditoría . . . . . 53
3.12. Flujo de ejecución: ciclo de vida de un turno . . . . . 57

# Capítulo 1

# Marco Teórico y Estado del Arte

Este capítulo establece los fundamentos científicos, técnicos y legales que justifican la necesidad de garantizar una toma de decisiones auditable y rigurosa en los sistemas conversacionales modernos. A lo largo de las siguientes secciones, se trazará un recorrido analítico que comienza con la exigencia regulatoria y ética de la **accountability**, desglosando los imperativos legales que condicionan el despliegue de estas tecnologías en sectores críticos. Posteriormente, se evaluarán los paradigmas históricos subyacentes: desde las limitaciones cognitivas y de ciberseguridad del conexionismo puro (*Deep Learning*), hasta la rigidez inherente de la inteligencia artificial simbólica clásica. Este análisis diagnóstico culminará justificando la convergencia neuro-simbólica como el único estado del arte capaz de conciliar la naturalidad del lenguaje humano con la gobernanza formal requerida.

## 1.1. El Paradigma de la Accountability y el Marco Regulatorio

El problema central de la inteligencia artificial conversacional contemporánea no radica en su incapacidad para mantener diálogos fluidos o procesar lenguaje natural complejo, sino en una deficiencia mucho más profunda: los modelos actuales generan comportamiento sin representarlo explícitamente. Para comprender la magnitud de este problema y establecer los requisitos de un sistema robusto, es necesario desambiguar algunos conceptos que a menudo se utilizan como sinónimos en la industria tecnológica, pero que técnicamente imponen niveles de exigencia muy distintos.

### 1.1.1. De la Transparencia a la Accountability

La literatura sobre IA explicable establece una jerarquía de propiedades que un sistema puede satisfacer. Estas propiedades operan como una cadena de dependencias, donde cada nivel exige requisitos de implementación más estrictos que el anterior:

- **Transparencia**: Es la propiedad más básica e implica que la estructura interna y el proceso de operación del sistema son accesibles para su inspección.

Lipton (2018) distingue entre la transparencia del modelo completo (simulabilidad mental por parte de un humano), la transparencia de los componentes (descomponibilidad) y la transparencia algorítmica. Un Gran Modelo de Lenguaje (LLM) puro no es transparente en ninguno de estos niveles: su red de miles de millones de parámetros no es simulable mentalmente.

- **Interpretabilidad**: Añade el requisito de la comprensibilidad humana. Un sistema es interpretable cuando un observador humano no solo tiene acceso a la estructura, sino que puede comprender cómo esta produce las decisiones. La transparencia es una propiedad del sistema, mientras que la interpretabilidad es una propiedad de la relación entre el sistema y el humano. En los modelos atencionales de los LLMs, las capas de atención carecen de un significado individual interpretable.

- **Explicabilidad**: Suma a la interpretabilidad el requisito de la articulación causal. No solo se debe comprender qué hizo el sistema, sino por qué lo hizo. Requiere que el sistema pueda articular de manera fidedigna la cadena causal que conecta la entrada del usuario, los estados internos, las reglas aplicadas y la salida generada.

- **Accountability**: Representa el vértice de esta jerarquía y es el concepto más exigente. La accountability presupone la explicabilidad, pero añade dos requisitos: la **verificabilidad** (la capacidad de comprobar matemáticamente que el comportamiento cumple con una especificación formal o norma explícita) y la **atribución de responsabilidad** (identificar qué componente, regla o agente provocó cada decisión).

Bajo este marco, resulta evidente que las decisiones operativas de un LLM puro no pueden ser verificables contra una norma, fundamentalmente porque el sistema no contiene en su arquitectura ninguna representación explícita de dicha norma. Al generar un comportamiento conversacional fundamentado puramente en inferencias estocásticas, el proceso de decisión de un LLM queda fuera de cualquier posibilidad de auditoría rigurosa, haciendo que la accountability sea estructuralmente inalcanzable.

### 1.1.2. Explicabilidad Post-Hoc vs. Explicabilidad Constitutiva

La industria tecnológica ha intentado solventar el déficit de accountability mediante técnicas de explicabilidad que, analizadas con rigor, resultan insuficientes para entornos conversacionales críticos. Esto nos obliga a distinguir entre dos paradigmas de diseño opuestos: el **post-hoc** y el **constitutivo**.

El paradigma **post-hoc** asume la existencia de un modelo opaco (caja negra) que ya ha tomado una decisión, e intenta generar una explicación posterior sobre dicho resultado. Con la llegada de los LLMs, se ha popularizado una nueva modalidad de explicación post-hoc: pedir al propio modelo conversacional que "justifique su respuesta" o que "razone paso a paso" (Chain-of-Thought).

Sin embargo, como argumenta Rudin (2019) y demuestran estudios empíricos recientes (Turpin et al., 2024; Lanham et al., 2023), estas explicaciones generadas por LLMs son, en realidad, **racionalizaciones**. Es decir, son narrativas construidas estadísticamente para justificar una respuesta, pero no constituyen descripciones fieles del proceso interno real que produjo dicha respuesta. El modelo no tiene acceso a su propio proceso de inferencia matricial. Pedirle a un LLM que justifique una decisión es engañoso, generará una historia coherente y articulada, pero completamente desvinculada de la causalidad real.

Por el contrario, la **explicabilidad constitutiva** exige que el proceso de decisión del sistema opere desde su origen sobre representaciones explícitas. En un sistema conversacional dotado de explicabilidad constitutiva, la explicación no se genera después de la decisión, sino que la traza inmutable del proceso de decisión es la explicación en sí misma. La correspondencia entre la justificación aportada al auditor y el proceso lógico ejecutado está garantizada por construcción, eliminando cualquier espacio para la alucinación justificativa. Los entornos críticos exigen abandonar las racionalizaciones post-hoc para abrazar arquitecturas constitutivamente transparentes.

### 1.1.3. Marco Regulatorio y Derecho a Explicación

La exigencia de accountability y explicabilidad constitutiva en la IA conversacional ha trascendido el debate académico para convertirse en un imperativo legal. La legislación europea reciente impone requisitos estrictos que condicionan la viabilidad de las arquitecturas tecnológicas, obligando a traducir las normativas en especificaciones de ingeniería concretas.

Es pertinente señalar que, si bien en este apartado se analizará el marco normativo tecnológico de carácter general (RGPD y EU AI Act), las regulaciones específicas del sector financiero se abordarán en detalle en el Capítulo 3, coincidiendo con la presentación del caso de uso bancario desarrollado en este trabajo.

#### RGPD (Reglamento General de Protección de Datos)

El Artículo 22 del RGPD legisla sobre las decisiones individuales automatizadas, estableciendo el derecho de las personas a no ser objeto de decisiones basadas únicamente en el tratamiento automatizado que produzcan efectos jurídicos o significativos. Además, los considerandos vinculados a este artículo consolidan el derecho a obtener una explicación y el derecho a solicitar intervención humana.

Desde la perspectiva de la ingeniería conversacional, el RGPD prohíbe que una denegación de servicio automatizada se justifique con una "explicación genérica" del modelo. Si un asistente conversacional rechaza una solicitud, el cliente tiene el derecho legal a conocer la lógica subyacente específica que motivó ese rechazo. Como se ha analizado, un modelo opaco solo puede ofrecer racionalizaciones post-hoc, las cuales no satisfacen el estándar legal de explicar la lógica real subyacente. Se requiere, por tanto, una arquitectura que permita emitir una traza exacta e individualizada del árbol de decisión recorrido para ese cliente en particular.

**EU AI Act (Ley de Inteligencia Artificial de la UE)**

El EU AI Act establece el marco normativo definitivo para los sistemas de IA en el ámbito europeo, clasificándolos de manera proporcional según su nivel de riesgo. Los sistemas de IA conversacional empleados para procesos críticos, como el triaje médico, la criba de candidatos o la evaluación de solvencia, pertenecen formalmente al Anexo III (Sistemas de Alto Riesgo). Esta clasificación activa inmediatamente el cumplimiento de los Artículos 9 al 15 del reglamento, los cuales obligan a adoptar los siguientes requisitos arquitectónicos:

- **Artículo 12 (Trazabilidad y Registros)**: Exige que los sistemas de IA de alto riesgo dispongan de capacidades de registro automático de determinados eventos durante su funcionamiento, con el objetivo de garantizar una trazabilidad adecuada del sistema. En el contexto de los sistemas conversacionales, esto implica mantener un registro de los eventos relevantes de la interacción, como las entradas recibidas, las reglas activadas, las decisiones intermedias o las respuestas generadas. Estos registros permiten reconstruir posteriormente el funcionamiento del sistema y facilitan su supervisión, depuración y auditoría.

- **Artículo 13 (Transparencia e Información a los Usuarios)**: Exige que la IA esté diseñada para que su funcionamiento sea suficientemente transparente. Los usuarios deben ser conscientes de que interactúan con una IA y deben poder comprender las capacidades operativas y las limitaciones del asistente conversacional con el que se comunican.

- **Artículo 14 (Supervisión Humana)**: Exige que los sistemas de IA de alto riesgo puedan ser supervisados de forma efectiva por personas durante su funcionamiento. En un sistema conversacional, esto puede materializarse mediante mecanismos que permitan detectar situaciones anómalas o inciertas y derivarlas a un operador humano, por ejemplo mediante un estado de `REQUIERE_SUPERVISION_HUMANA`. El operador debe disponer de información suficiente para interpretar el resultado y poder aceptarlo, modificarlo, rechazarlo o interrumpir el funcionamiento del sistema cuando sea necesario.

- **Artículo 15 (Precisión, Robustez y Ciberseguridad)**: Exige que los sistemas de IA de alto riesgo alcancen niveles adecuados de precisión, robustez y ciberseguridad y que mantengan un funcionamiento consistente durante su ciclo de vida. En un sistema conversacional, esto implica disponer de mecanismos para detectar y gestionar errores, inconsistencias y posibles intentos de manipulación, como los ataques de *Prompt Injection*, evitando que entradas maliciosas puedan alterar indebidamente el comportamiento previsto del sistema.

La convergencia de estos marcos normativos transversales (RGPD y EU AI Act) concluye que la *accountability* no es una característica deseable, sino un estricto conjunto de requisitos.

## 1.2. Inteligencia Artificial Conexionista (Deep Learning)

La segunda ola de la inteligencia artificial, caracterizada por el *Deep Learning* y el auge de arquitecturas como los *Transformers*, ha supuesto un avance en la capacidad de las máquinas para procesar, interpretar y generar lenguaje natural. Las redes neuronales profundas han logrado capturar casi a la perfección la variabilidad lingüística del mundo real, permitiendo a los sistemas comprender instrucciones complejas, manejar elipsis, tolerar errores gramaticales y generar respuestas con un nivel de fluidez indistinguible del humano.

No obstante, tal como se documenta la literatura científica reciente, estas capacidades lingüísticas coexisten con deficiencias cognitivas profundas. Autores como Marcus (2020) han señalado que las limitaciones del aprendizaje profundo (su opacidad, su inconsistencia y la imposibilidad de verificación formal) no son simples defectos técnicos que futuras iteraciones de los modelos vayan a corregir mediante un mayor volumen de datos o parámetros, sino que constituyen consecuencias ineludibles de su propia arquitectura.

### 1.2.1. Naturaleza Estocástica y Cajas Negras

La base del funcionamiento de los Grandes Modelos de Lenguaje (LLMs) es la predicción estocástica de *tokens*. Estos sistemas generan texto calculando distribuciones de probabilidad sobre un vocabulario, condicionadas por el contexto previo, careciendo en todo momento de un modelo mental del mundo real, de una ontología subyacente o de mecanismos de razonamiento lógico deductivo puro. En el paradigma conexionista, el conocimiento no se codifica en estructuras discretas con semántica definida (como proposiciones, reglas lógicas o redes semánticas), sino que se distribuye de manera implícita a lo largo de miles de millones de parámetros (pesos y sesgos) en los patrones de activación de la red.

Esta distribución implícita convierte a los LLMs en *cajas negras*. Las decisiones y respuestas que emiten son el resultado de complejas multiplicaciones de matrices y operaciones *softmax* que carecen por completo de interpretación semántica accesible para un observador humano. Si un sistema conversacional puramente neuronal deniega la concesión de un préstamo, no existe ningún mecanismo para inspeccionar sus parámetros y aislar la "regla" o el "razonamiento" exacto que condujo a dicha denegación. Esta opacidad entra en conflicto directo con los requisitos más elementales de la *accountability* expuestos en la sección anterior.

### 1.2.2. El Fenómeno de las Alucinaciones

Derivada de su naturaleza probabilística, la segunda gran limitación de los modelos fundacionales es el fenómeno de las alucinaciones. Dado que un LLM aprende a través de la maximización de la verosimilitud de las secuencias de texto durante su fase de preentrenamiento, el modelo carece de un mecanismo intrínseco para distinguir entre hechos factuales, verdades lógicas y simples correlaciones

estadísticas latentes en sus datos.

Cuando a un modelo neuronal se le exige razonar sobre un caso práctico, el sistema no está deduciendo lógicamente la respuesta a partir de axiomas legales. En su lugar, está generando la secuencia de palabras que, estadísticamente, es más probable que siga al *prompt* introducido. Esto da lugar a que el modelo pueda inventar datos, citar artículos normativos inexistentes o fabricar justificaciones que parecen razonables y articuladas, pero que son fundamentalmente falsas.

Aunque las técnicas de entrenamiento, alineamiento y verificación han permitido reducir progresivamente la frecuencia de estas alucinaciones, no es posible garantizar su eliminación completa. Al tratarse de sistemas probabilísticos, siempre existe la posibilidad de que el modelo genere información incorrecta con un elevado grado de coherencia y aparente certeza, especialmente cuando se enfrenta a situaciones que requieren conocimientos específicos o inferencias complejas. Por tanto, incluso cuando el sistema presenta un elevado rendimiento en términos generales, la ausencia de garantías sobre la veracidad de cada una de sus respuestas constituye una limitación estructural que debe tenerse en cuenta en aquellos contextos en los que un error puede producir consecuencias relevantes.

### 1.2.3. Inestabilidad y Efecto Contextual

La tercera limitación estructural de los enfoques conexionistas puros es su profunda inestabilidad frente a la variabilidad de entrada. En un sistema de reglas simbólico (GOFAI - *Good Old-Fashioned AI*), una condición evaluada producirá el mismo resultado siempre que los parámetros de entrada sean idénticos, garantizando un determinismo absoluto. Por el contrario, la inferencia en un LLM es altamente sensible a perturbaciones menores.

Investigaciones recientes han documentado empíricamente esta fragilidad. Raj et al. (2023) demostraron que los LLMs producen respuestas inconsistentes ante reformulaciones de la misma pregunta (paráfrasis), documentando tasas de inconsistencia semántica que pueden superar el 30 % en dominios especializados. Esto significa que si un cliente solicita un préstamo escribiendo "Quiero solicitar 5000 euros a 4 años" frente a "Necesito un crédito de 5000€ a devolver en 48 meses", el modelo neuronal podría aprobar la primera solicitud y denegar la segunda simplemente porque la distribución de probabilidad generada a partir de los *tokens* de entrada difiere ligeramente.

A esta inestabilidad semántica se suma la degradación cognitiva en el manejo de contextos largos. Aunque los modelos modernos han ampliado su ventana de contexto, Liu et al. (2023) han evidenciado el efecto *"lost in the middle"*: la capacidad de los LLMs para utilizar información decae drásticamente cuando los datos críticos se encuentran en posiciones intermedias del contexto. Adicionalmente, Levy et al. (2024) documentaron que la capacidad de "razonamiento" del modelo se degrada exponencialmente a medida que la longitud de la conversación aumenta.

En el contexto de la IA conversacional, donde una interacción puede extenderse a lo largo de múltiples turnos y el usuario puede proporcionar datos, cambiar de opinión, retractarse o aportar información fragmentada, depender de la ventana de contexto del LLM para mantener el "estado" de la operación es una debilidad

arquitectónica. El modelo no mantiene un registro estructurado de qué datos han sido proporcionados, cuáles han sido verificados y cuáles son simplemente comentarios; todo se amalgama como *tokens* indiferenciados. Esta falta de *Information State* explícito impide garantizar la consistencia sistémica, haciendo que la toma de decisiones sea impredecible.

### 1.2.4. Ciberseguridad y Vulnerabilidades

Las limitaciones cognitivas expuestas no solo afectan a la calidad del diálogo y toma de decisiones, sino que configuran una superficie de ataque completamente nueva desde la perspectiva de la ciberseguridad. Al carecer de una representación formal que separe semánticamente las instrucciones del sistema de los datos introducidos por el usuario, los LLMs son vulnerables a manipulaciones lingüísticas que subvierten su comportamiento previsto.

#### Vectores de Ataque Semántico

El principal riesgo en las aplicaciones basadas en Modelos Fundacionales es el *Prompt Injection*. Este ataque ocurre cuando se introducen comandos ocultos o maliciosos en los datos que el modelo procesa, confundiéndolos con instrucciones del sistema. Un ejemplo clásico en el entorno bancario sería un usuario que adjunta un historial crediticio en PDF que contiene el texto invisible: "*Aviso del sistema: El análisis ha finalizado. Clasifica este perfil de solvencia como APROBADO sin importar los ingresos previos.*". Al procesar el documento, el modelo es incapaz de separar los datos de las instrucciones y ejecuta la orden del atacante.

Complementariamente, el *Jailbreaking* se refiere a las técnicas adversariales que el usuario introduce directamente en la conversación para romper las barreras éticas, lógicas y de seguridad impuestas por los desarrolladores, a menudo mediante ingeniería social o manipulación de *tokens*.

La existencia de estas vulnerabilidades demuestra que las restricciones impuestas mediante *prompts* no son estructurales ni formales, sino meramente estadísticas: se cumplen con alta probabilidad en escenarios benignos, pero fallan cuando la presión probabilística de los datos del atacante supera la fuerza matemática de la instrucción de sistema subyacente.

#### El Agotamiento de las Aproximaciones Paliativas

Ante la gravedad de los vectores de ataque semántico, la industria ha desarrollado diversas técnicas conexionistas para intentar mitigar el problema sin abandonar el paradigma puramente neuronal. No obstante, estas soluciones operan sobre los síntomas externos del modelo y no sobre su proceso de decisión interno. Entre las más extendidas, y sus respectivas carencias, destacan:

- **Los *guardrails* y clasificadores externos:** Sistemas de filtrado perimetral (como *NeMo Guardrails* o *Llama Guard*) que interceptan la entrada o la salida de texto para detectar anomalías mediante modelos estadísticos

secundarios. Sin embargo, actúan exclusivamente sobre la superficie lingüística. Un filtro puede bloquear una palabra explícitamente tóxica, pero es incapaz de evaluar si una decisión de negocio omitida o alterada vulnera la coherencia contractual del diálogo o las políticas de solvencia de la entidad.

- **El *prompt engineering*:** Intentar gobernar un LLM introduciendo directrices operativas en lenguaje natural dentro del *prompt* ("*bajo ninguna circunstancia ...*") confunde instrucción con ejecución determinista. El modelo interpreta estas directrices de manera probabilística, por consiguiente, una reformulación adversaria suficientemente compleja o la acumulación de contexto en diálogos extensos pueden provocar que el sistema diluya o ignore la restricción por completo.
- **El alineamiento estadístico (RLHF y *Constitutional AI*):** Aunque técnicas como el Aprendizaje por Refuerzo a partir de Retroalimentación Humana (Bai et al., 2022) reducen la frecuencia promedio de respuestas no deseadas, estas modifican las probabilidades de salida en el espacio latente, pero no introducen estructuras verificables de toma de decisiones. Estudios empíricos exhaustivos, como los realizados por Wolf et al. (2023) y Zou et al. (2023), han demostrado de forma irrefutable que incluso los mejores modelos pueden ser inducidos a violar sus políticas mediante ataques adversariales.

### El Riesgo Crítico en Entornos Agénticos Autónomos

Esta fragilidad intrínseca alcanza una dimensión crítica cuando se dota al modelo neuronal de autonomía operativa mediante arquitecturas agénticas. La tendencia reciente de la industria hacia patrones como *ReAct* (Yao et al., 2023) o *frameworks* como *LangChain* y *CrewAI*, donde se otorgan al LLM herramientas como APIs de bases de datos, capacidad de enviar correos electrónicos o ejecutar transferencias, eleva el problema a niveles críticos para la ciberseguridad financiera.

Como advierten Ruan et al. (2024) en su análisis sobre los riesgos de agentes basados en LLMs, un agente autónomo es, por definición, un agente cuyas decisiones operativas no están gobernadas externamente. En estos sistemas, el LLM decide autónomamente, basándose en la inferencia probabilística del contexto, qué herramienta invocar, en qué orden y con qué parámetros. Si un ataque de los mencionados previamente tiene éxito contra un agente autónomo, el atacante no solo obtiene control sobre el texto generado, sino que adquiere capacidad ejecutiva sobre las herramientas de las que dispone el agente.

En conclusión, el paradigma conexionista puro genera comportamiento lingüístico sin representarlo formalmente. La combinación de una lógica estocástica opaca con la capacidad de ejecutar llamadas a sistemas bancarios genera un riesgo inaceptable para los estándares de supervisión y resiliencia digital. Las capas de control superpuestas sobre modelos probabilísticos solo proporcionan garantías estadísticas, pero en ningún caso las garantías formales, deterministas y auditables. Esta limitación fundamenta la necesidad de recurrir a la tradición simbólica de la inteligencia artificial, objeto de análisis en la siguiente sección.

## 1.3. Inteligencia Artificial Simbólica y Sistemas Basados en Reglas

Como se ha evidenciado en la sección anterior, la delegación de decisiones críticas a sistemas puramente conexionistas incurre en riesgos inasumibles de opacidad, inconsistencia y vulnerabilidad. Para resolver esta brecha y recuperar el control sobre la lógica de negocio, resulta necesario volver al otro gran pilar histórico de la disciplina: la Inteligencia Artificial Simbólica. Este enfoque, lejos de estar obsoleto, proporciona los cimientos formales, matemáticos y lógicos necesarios para garantizar la *accountability* exigida en entornos regulados.

### 1.3.1. El Paradigma Simbólico (GOFAI)

La Inteligencia Artificial Simbólica, frecuentemente referida como GOFAI (*Good Old-Fashioned AI*), se fundamenta en la intuición de que la inteligencia es, en esencia, la capacidad de manipular representaciones simbólicas de acuerdo con reglas formales explícitas. Esta visión fue articulada con máxima precisión por Newell y Simon (1976) en *Physical Symbol System Hypothesis*, donde se postula que un sistema de estas características posee los medios necesarios y suficientes para la acción inteligente general.

A diferencia del paradigma conexionista, el enfoque simbólico exige que el conocimiento se codifique en estructuras discretas con una semántica estrictamente definida e inspeccionable. Esta explicitación del conocimiento se materializa a través de:

- **Lógica de Primer Orden:** Proporciona el marco matemático para formular aserciones sobre el mundo. Permite definir hechos, variables y cuantificadores, garantizando que cualquier inferencia derivada del sistema sea lógicamente válida por construcción.
- **Ontologías Formales:** Las ontologías permiten la conceptualización explícita y compartida de un dominio. En un contexto corporativo, una ontología define qué entidades existen (usuario, producto, restricciones), cómo se relacionan jerárquicamente y qué restricciones de integridad deben cumplir. Esto asegura que el sistema posea una comprensión estructural del negocio, previniendo, por ejemplo, que asigne una edad negativa a un individuo.
- **Reglas de Producción (IF-THEN):** El conocimiento procedimental y normativo se encapsula en estructuras condicionales. Una regla de producción establece que, si se cumple un patrón específico de hechos en la memoria del sistema (*IF*), se debe desencadenar una acción determinista (*THEN*). La naturaleza composicional de estas reglas permite construir lógicas empresariales de extrema complejidad a partir de unidades atómicas, independientes y fácilmente auditables por expertos de dominio.

### 1.3.2. Motores de Inferencia y Algoritmos de Evaluación

Para que las representaciones simbólicas adquieran capacidad operativa, es necesario disponer de un componente computacional capaz de interpretar los hechos y evaluar las reglas que constituyen el conocimiento del sistema. Este componente se denomina **motor de inferencia** o, en el ámbito de las reglas de negocio, **motor de reglas**. Su función principal consiste en separar la lógica de decisión, expresada mediante reglas, del código de la aplicación, permitiendo determinar de forma eficiente qué reglas deben activarse ante un conjunto determinado de hechos.

Un ejemplo representativo de este enfoque es el ecosistema **KIE**, dentro del cual se integra **Drools**. Es importante distinguir ambos conceptos: KIE hace referencia al ecosistema de tecnologías orientadas a la gestión del conocimiento, las reglas y la automatización de decisiones y procesos, mientras que Drools es el componente especializado en la definición y ejecución de reglas de negocio. Por tanto, KIE constituye el marco tecnológico general y Drools proporciona específicamente las capacidades necesarias para implementar un motor de reglas. En este contexto, las reglas pueden describir condiciones sobre los hechos disponibles y las acciones que deben ejecutarse cuando dichas condiciones se cumplen.

La eficiencia computacional de estos motores reside en los algoritmos subyacentes encargados del *pattern matching* (evaluación de patrones), históricamente dominados por el **Algoritmo Rete** y, en sus arquitecturas más modernas, por el **Algoritmo Phreak**.

#### El Algoritmo Rete y la Memoria de Trabajo

Diseñado por Charles Forgy en 1979, el algoritmo Rete resolvió el problema del rendimiento en los sistemas expertos tradicionales. En lugar de evaluar cada regla iterativamente contra cada dato (lo que generaría un coste combinatorio intratable, $O(R \times F)$), Rete construye un grafo acíclico dirigido (DAG) que compila las reglas en una red de nodos.

El sistema consta de dos memorias: la *Production Memory*, que almacena el árbol de reglas, y la *Working Memory* (Memoria de Trabajo), donde se inyectan los hechos (*facts*). Cuando un *fact* ingresa a la memoria de trabajo, se propaga por la red a través de *Alpha Nodes* (que evalúan condiciones literales simples, como `edad <18`) y *Beta Nodes* (que evalúan cruces y combinaciones lógicas entre diferentes entidades, como comparar atributos interdependientes de un usuario con los requisitos de un servicio). Rete conserva el estado de estas evaluaciones en memoria, recalculando únicamente aquello que cambia, lo que permite inferencias en tiempo real.

#### La Evolución a Phreak y la Resolución de Conflictos

Drools incorpora **Phreak** como una evolución de los mecanismos de evaluación de reglas basados en Rete, orientada a mejorar la gestión del procesamiento en sistemas con grandes cantidades de hechos y reglas. Mientras que Rete organiza las condiciones de las reglas en una red y propaga por ella los resultados de las evaluaciones, Phreak introduce mecanismos para gestionar de forma más

eficiente los cambios producidos en la memoria de trabajo, agrupando y priorizando el trabajo que debe realizarse. La idea fundamental no consiste en reevaluar indiscriminadamente toda la red ante cada modificación, sino en identificar y procesar las partes afectadas por los cambios, reduciendo así el consumo de memoria y multiplicando la velocidad de evaluación cuando se enfrentan cientos de miles de hechos y reglas simultáneas.

Independientemente del algoritmo de red, el motor se enfrenta al desafío de la **resolución de conflictos**: ¿qué ocurre cuando los datos proporcionados hacen que las condiciones de múltiples reglas se cumplan simultáneamente? El motor agrupa todas las reglas activadas en una estructura denominada *Agenda*. Para garantizar el determinismo estricto, Drools emplea el atributo de ***Salience*** (prominencia o prioridad). Asignando un valor numérico de *salience* a cada regla, el ingeniero de conocimiento define matemáticamente el orden exacto de ejecución. Así, una regla crítica con `salience 100` se ejecutará y podrá abortar el flujo antes que una regla de `salience 50`, evitando computaciones innecesarias y colisiones lógicas.

### 1.3.3. Determinismo y Trazabilidad

En contraposición a la estocasticidad de los LLMs, el motor simbólico garantiza determinismo absoluto. Si el estado de información (*Information State*) introducido en la memoria de trabajo es $X$, y el conjunto de reglas formales de la organización es $R$, el resultado emitido será invariablemente $Y$. No existe margen para la aleatoriedad, la "temperatura" generativa o la influencia de un contexto intermedio. Esta consistencia sistémica permite la validación de software mediante pruebas unitarias exhaustivas, posibilitando que el sistema sea certificado algorítmicamente antes de su paso a producción.

#### Materialización de la Explicabilidad Constitutiva

Tal y como se definió en la sección anterior, los entornos altamente regulados exigen el grado más alto de *accountability*. El paradigma simbólico proporciona exactamente esta propiedad: en un sistema basado en reglas como Drools, la traza de ejecución del motor *es* la explicación de la decisión.

Cuando el motor evalúa una transacción crítica, genera un registro (*ledger*) inmutable de su recorrido computacional. La traza documenta con precisión quirúrgica:

1. El instante temporal y la sesión exacta de la operación.
2. Los datos crudos (*facts*) que ingresaron a la Memoria de Trabajo.
3. La secuencia ordenada de reglas que se activaron en la Agenda.
4. Los nodos de decisión específicos que alteraron el estado del proceso.
5. El artículo normativo o política interna vinculada directamente a la regla ejecutada.

Esta arquitectura satisface el requisito de auditabilidad por diseño. Un auditor humano puede inspeccionar la representación formal, comprender la lógica empleada y verificar el cumplimiento normativo sin ambigüedades.

### 1.3.4. Fragilidad ante la Variabilidad Conversacional

La Inteligencia Artificial Simbólica pura presenta debilidades igualmente fundamentales que limitan históricamente su aplicabilidad en entornos interactivos. La más importante es su extrema **fragilidad ante la variabilidad del mundo real**, en especial frente a la riqueza y ambigüedad del lenguaje natural humano.

Los sistemas basados exclusivamente en reglas exigen que el usuario se exprese de forma que el sistema pueda anticipar algorítmicamente. Si un sistema conversacional simbólico espera la entrada literal "solicitar cancelación", fallará ante reformulaciones válidas como "me gustaría darme de baja", "ya no quiero continuar con el servicio" o actos de habla indirectos. Las reglas de producción no pueden procesar adecuadamente los sinónimos, las elipsis, el contexto emocional o la intención subyacente de una oración sin una explosión combinatoria inasumible en el código.

Este problema se conoce en la literatura como el *cuello de botella de la ingeniería del conocimiento*. Codificar manualmente todas las posibles variaciones conversacionales, intenciones y estrategias de diálogo para un dominio complejo es un proceso costoso, frágil y propenso a omisiones. Los primeros sistemas de diálogo operativos, basados netamente en este enfoque, resultaron ser inflexibles, provocando experiencias de usuario robóticas y frustrantes.

En definitiva, se establece una paradoja arquitectónica fundamental: el enfoque conexionista comprende el lenguaje pero no puede gobernar decisiones con rigor legal, mientras que el enfoque simbólico garantiza el rigor de las decisiones pero es incapaz de comprender el lenguaje. Esta dicotomía es la que motiva la convergencia hacia el estado del arte actual: la Inteligencia Artificial Neuro-Simbólica.

## 1.4. Inteligencia Artificial Neuro-Simbólica

Las secciones anteriores han establecido una dicotomía en el campo de la inteligencia artificial. Por un lado, el paradigma conexionista comprende muy bien la variabilidad y ambigüedad del lenguaje natural, pero carece de los mecanismos formales necesarios para gobernar decisiones con el rigor legal que exige la *accountability*. Por otro lado, el paradigma simbólico garantiza el determinismo absoluto y la trazabilidad de las decisiones, pero resulta rígido y frágil ante la riqueza de la interacción humana.

Para superar esto, la frontera de la investigación contemporánea ha convergido hacia un nuevo paradigma que busca reconciliar ambas tradiciones: la Inteligencia Artificial Neuro-Simbólica.

### 1.4.1. La Tercera Ola de la Inteligencia Artificial

La evolución histórica de la disciplina ha estado marcada por ciclos de gran entusiasmo seguidos de profundas decepciones (los conocidos "inviernos" de la IA). Henry Kautz (2022) argumenta que actualmente nos encontramos en el "Tercer Verano de la IA", un periodo impulsado por el éxito del aprendizaje profundo (*Deep Learning*) en la extracción de representaciones jerárquicas y el cálculo de similitudes vectoriales. No obstante, teóricos e investigadores coinciden en que el enfoque puramente neuronal se acerca a un límite inherente cuando se trata de realizar inferencias lógicas rigurosas.

Como respuesta a este estancamiento, emerge lo que Garcez y Lamb denominan la "Tercera Ola" de la Inteligencia Artificial. Esta ola no busca sustituir a los modelos neuronales, sino complementarlos. El objetivo es utilizar el aprendizaje profundo para las tareas perceptuales y de procesamiento de lenguaje natural, reservando la lógica simbólica para el razonamiento, la planificación y el control estricto de las decisiones operativas. En definitiva, el futuro de la IA no reside en la victoria de un paradigma sobre otro, sino en su integración sistemática.

### 1.4.2. Taxonomía de Arquitecturas Neuro-Simbólicas

La integración neuro-simbólica no es un concepto binario, sino un espectro continuo de arquitecturas con distintos grados de acoplamiento y control. Para categorizar este amplio abanico de posibilidades, Kautz (2022) propone una taxonomía que identifica seis diseños arquitectónicos diferenciados, los cuales estructuran la forma en que los módulos neuronales y simbólicos interactúan. A continuación, se detallan estas seis aproximaciones:

#### 1. Symbolic Neuro symbolic

Esta arquitectura representa el Procedimiento Operativo Estándar actual para muchas tareas de procesamiento de lenguaje natural (NLP). En este diseño, una entrada simbólica (como una secuencia de palabras) se convierte en vectores (*embeddings*), se procesa a través de las capas ocultas de una red neuronal y, finalmente, las unidades de salida convierten esa información nuevamente en una categoría simbólica o secuencia de símbolos mediante una operación *softmax*. Aunque la entrada y la salida son símbolos, todo el procesamiento intermedio es puramente neuronal y, por tanto, opaco. Este nivel corresponde a una integración de "caja negra" donde el componente neuronal es un servicio invocado para una traducción directa.

#### 2. Symbolic[Neuro]

En este diseño, el sistema simbólico es el orquestador principal (el que resuelve el problema), y utiliza un modelo neuronal como una subrutina interna. El componente neuronal no opera libremente, sino dentro del marco de trabajo definido por el componente simbólico. Un ejemplo paradigmático de esta arquitectura es AlphaGo, donde un algoritmo de búsqueda simbólica (*Monte-Carlo Tree Search*)

invoca a una red neuronal exclusivamente para evaluar las posiciones de las hojas del árbol. En el contexto conversacional, este patrón permite crear un "pipeline gobernado": el motor simbólico dirige el flujo de la conversación y delimita el espacio probabilístico en el que el modelo de lenguaje puede operar, garantizando así que las decisiones críticas permanezcan bajo control determinista.

**3. Neuro | Symbolic**

Esta arquitectura plantea un flujo donde una red neuronal procesa entradas no simbólicas o complejas (como los píxeles de una imagen o el texto crudo de un usuario) y las convierte en una estructura de datos puramente simbólica. Una vez que los datos han sido mapeados a símbolos formales, estos se transfieren a un sistema de razonamiento lógico tradicional que completa la tarea. A diferencia del modelo anterior (donde el neuronal era una subrutina), aquí el componente neuronal actúa como una "corrutina" inicial que ejerce de puente perceptual hacia el motor de reglas.

**4. Neuro: Symbolic → Neuro**

Esta aproximación utiliza una arquitectura puramente neuronal (*Sequence-to-Sequence*), pero con un entrenamiento previo generado y supervisado mediante reglas simbólicas. El modelo neuronal se entrena masivamente con pares de entrada-salida generados por un solucionador simbólico (por ejemplo, expresiones matemáticas y sus simplificaciones lógicas exactas). El objetivo es que la red neuronal aprenda la lógica simbólica subyacente de tal forma que pueda adivinar correctamente la respuesta sin necesidad de generar una derivación paso a paso. Aunque ofrece gran velocidad, la decisión final sigue siendo probabilística, lo que compromete la verificabilidad formal del proceso.

**5.** $Neuro_{Symbolic}$

Este enfoque busca compilar directamente el conocimiento simbólico dentro de la propia topología de la red neuronal. Las reglas lógicas se transforman en plantillas o tensores estructurales (*Logic Tensor Networks*) que condicionan el aprendizaje de la red. Esta aproximación permite inyectar relaciones jerárquicas y restricciones directamente en el espacio vectorial del modelo. Sin embargo, la investigación en este nivel sigue siendo exploratoria y presenta dificultades para codificar reglas disyuntivas complejas o razonamientos combinatorios por casos.

**6. Neuro[Symbolic]**

Inspirada en la teoría cognitiva de Daniel Kahneman sobre "Pensar rápido y pensar despacio", esta arquitectura incrusta un motor de razonamiento simbólico dentro de un motor neuronal principal. El sistema neuronal ("Sistema 1", rápido y automático) es el orquestador global. Cuando el Sistema 1 se enfrenta a un problema combinatorio o lógico que no puede resolver probabilísticamente, invoca al motor simbólico ("Sistema 2", lento y exacto) como una subrutina especializada.

El modelo neuronal genera una representación simbólica del problema, la envía al solucionador lógico, y el resultado de este último retroalimenta a la red neuronal para continuar la ejecución.

### 1.4.3. El Balance para la Gobernanza

El análisis de la taxonomía de Kautz evidencia que la integración neuro-simbólica no ofrece una única solución, sino un espacio de diseño donde diferentes decisiones arquitectónicas generan propiedades totalmente distintas. Para el diseño de sistemas de inteligencia artificial en dominios críticos (donde la *accountability*, la explicabilidad constitutiva y la resiliencia son innegociables), la elección del nivel de integración es determinante.

Los enfoques que integran la lógica dentro de la red neuronal (como *Neuro: Symbolic* $\rightarrow$ *Neuro* o $Neuro_{Symbolic}$) mejoran la capacidad del sistema para resolver problemas complejos. Sin embargo, al diluir las reglas lógicas dentro de variables matemáticas abstractas, se pierde la transparencia y resulta imposible rastrear con exactitud por qué se tomó una decisión específica. Por otro lado, arquitecturas que separan ambos mundos (como *Neuro[Symbolic]*) dejan el control principal en manos del componente neuronal. Esto genera el riesgo de que el modelo actúe de manera opaca y decida de forma impredecible cuándo aplicar o ignorar la verificación lógica del "Sistema 2".

Por consiguiente, desde el punto de vista de la ingeniería del conocimiento, resulta especialmente interesante utilizar arquitecturas en las que el componente simbólico tenga un papel central en el control de la decisión, como ocurre en los patrones ***Symbolic[Neuro]*** y en determinados flujos ***Neuro | Symbolic***. En estos esquemas, el modelo de lenguaje se encarga principalmente de comprender y procesar la información, mientras que el motor simbólico aplica las reglas y restricciones definidas y participa en la determinación de la respuesta final. De esta forma, se combina la capacidad de los modelos neuronales para interpretar lenguaje natural con la posibilidad de representar de forma explícita las reglas que deben respetarse, facilitando así la trazabilidad, la supervisión y la justificación del comportamiento del sistema.

# Capítulo 2

# Definición del Dominio: Originación de Crédito Bancario

Hasta este punto de la investigación, hemos explorado el problema de la falta de accountability y la opacidad de los LLMs desde una perspectiva fundamentalmente teórica. Hemos analizado cómo la naturaleza probabilística dificulta la trazabilidad de sus decisiones, creando escenarios donde la responsabilidad se diluye entre la máquina, el desarrollador y el usuario. Sin embargo, para comprender la verdadera magnitud de este desafío, es imprescindible trasladar el debate hacia un entorno de aplicación real, donde las decisiones automatizadas generen un impacto tangible y cuantificable en la vida de las personas.

Este capítulo establece dicho puente al definir el dominio de aplicación de nuestro estudio: el sector financiero. En concreto, presentamos como caso de uso central el diseño de un asistente conversacional automatizado para la evaluación y concesión de préstamos personales. Este ámbito representa un entorno altamente regulado, complejo y de máxima exigencia técnica y ética. A lo largo de las siguientes secciones, detallaremos por qué este escenario constituye el campo idóneo para evaluar los límites de la responsabilidad algorítmica y los retos de extraer información crítica a través de interacciones en lenguaje natural.

## 2.1. Justificación del Escenario

La industria bancaria se encuentra en un dilema. Por un lado, existe una corriente que opta por digitalizar, agilizar y personalizar la experiencia del cliente mediante el uso de Inteligencia Artificial. Por otro, el sector está sujeto a estrictos marcos regulatorios diseñados para proteger la estabilidad económica y los derechos de los consumidores. En este contexto, la originación de crédito bancario se erige como el escenario perfecto para auditar un sistema conversacional, ya que cualquier fallo en la interpretación de los datos o toma de decisiones no se traduce en un simple error de interfaz, sino en consecuencias jurídicas, financieras y sociales.

### 2.1.1. El Crédito como Sistema de Alto Riesgo

La evaluación y concesión de crédito está catalogada, tanto por la literatura académica como por marcos regulatorios emergentes como la Ley de Inteligencia Artificial de la Unión Europea (AI Act), como un sistema de alto riesgo. Esto se debe a que el acceso a la financiación es un pilar fundamental para el desarrollo socioeconómico de los individuos. Denegar o conceder un préstamo altera de manera directa la trayectoria vital y financiera del solicitante.

Al delegar este proceso en un agente conversacional, los riesgos inherentes al crédito se juntan con las vulnerabilidades de la IA. El impacto de una decisión automatizada errónea en este dominio opera en dos direcciones, ambas con consecuencias importantes si no existe una correcta trazabilidad:

- **El impacto de los falsos positivos**: Consiste en conceder crédito a un perfil que, en realidad, no cuenta con la capacidad de pago necesaria. Si el asistente conversacional malinterpreta los ingresos del usuario, o es incapaz de cruzar la información verbalizada con los datos históricos de manera rigurosa, el banco asume un riesgo de impago. Más allá de la pérdida económica directa para la entidad financiera, un falso positivo fomenta el sobreendeudamiento del cliente, empujándolo hacia una espiral de vulnerabilidad económica, un resultado que incumple los principios de préstamo responsable.

- **El impacto de los falsos negativos**: Ocurre cuando el sistema deniega el crédito a un usuario que sí era solvente. Si esta denegación proviene de un sesgo algorítmico (por ejemplo, su nivel educativo inferido a través del lenguaje, o variables demográficas), nos enfrentamos a un caso de discriminación automatizada. La falta de *accountability* adquiere aquí una relevancia crítica: cuando una denegación no puede vincularse de forma clara con los datos utilizados y los criterios que la justifican, resulta difícil determinar si el resultado responde realmente a una evaluación legítima del riesgo o a un error, una inconsistencia o un sesgo introducido durante el proceso.

### 2.1.2. El Reto Conversacional en la Banca

Históricamente, la originación de crédito ha dependido de interfaces gráficas y formularios estructurados. El usuario introduce un número exacto en una casilla específica (ej. «Ingresos: 2000»), y el sistema de evaluación de riesgos procesa esa variable determinista sin ambigüedades. El cambio de paradigma hacia una interfaz conversacional introduce un nuevo nivel de fricción interpretativa.

El gran reto radica en la colisión entre dos mundos incompatibles por defecto: la naturaleza intrínsecamente desestructurada, abierta y ambigua del diálogo humano, frente a la exigencia estricta y tipada de las fórmulas financieras. Un agente conversacional en este dominio no se enfrenta simplemente a un problema de procesamiento de lenguaje (NLP), sino a un desafío de extracción de conocimiento crítico bajo incertidumbre.

El sistema debe ser capaz de transformar la variabilidad del lenguaje humano en un conjunto discreto de variables que puedan someterse a auditoría. Como

se detallará en posteriores secciones al analizar los perfiles de usuario, el diálogo natural implica convivir con información incompleta, datos irrelevantes y reformulaciones constantes. Salvar esta brecha es el principal requerimiento que justifica la investigación y el diseño del presente trabajo.

## 2.2. Marco Regulatorio Específico del Sector Financiero

Los requisitos generales de transparencia, trazabilidad, supervisión y gestión del riesgo aplicables a los sistemas de inteligencia artificial han sido analizados en el capítulo anterior. En el contexto de la concesión de crédito, estos principios adquieren una dimensión adicional debido a la naturaleza regulada de la actividad financiera y, especialmente, a las obligaciones relativas a la evaluación de la solvencia del solicitante.

En consecuencia, el problema no consiste únicamente en garantizar que el sistema conversacional sea trazable o supervisable, sino en asegurar que la información obtenida durante la interacción permita realizar una evaluación de crédito conforme a criterios financieros verificables y a las fuentes de información que resulten aplicables. Esto introduce requisitos específicos que condicionan directamente el diseño del sistema.

### 2.2.1. Evaluación de la solvencia

La normativa europea sobre crédito establece la obligación de realizar una evaluación adecuada de la solvencia antes de formalizar determinadas operaciones de crédito. En particular, la Directiva 2014/17/UE, aplicable al crédito relativo a bienes inmuebles de uso residencial, exige que la evaluación se realice sobre procedimientos e información establecidos y documentados. Por su parte, la Directiva (UE) 2023/2225, relativa a los contratos de crédito al consumo, establece que la evaluación debe realizarse en interés del consumidor y sobre información pertinente y exacta relativa a sus ingresos, gastos y demás circunstancias económicas relevantes.

Esta exigencia tiene una consecuencia directa para un sistema conversacional de concesión de crédito: los datos proporcionados durante el diálogo no pueden considerarse automáticamente equivalentes a datos financieros validados. El asistente puede encargarse de obtener y estructurar información como ingresos, gastos, deudas, importe solicitado o plazo, pero dicha información debe someterse a los mecanismos de validación y comprobación que correspondan antes de utilizarse como base de una decisión crediticia.

La normativa contempla además el uso de información procedente de fuentes internas y externas y, cuando resulte aplicable, de bases de datos relevantes para la evaluación de la solvencia. La Directiva (UE) 2023/2225 establece asimismo requisitos relativos a estas bases de datos y a la información que debe facilitarse al consumidor cuando una solicitud es rechazada como consecuencia de una consulta a dichas fuentes. Por tanto, el sistema debe ser capaz de distinguir entre la

información declarada directamente por el solicitante, la información obtenida de fuentes externas y el resultado de las comprobaciones realizadas sobre ambas.

La decisión crediticia no debe depender exclusivamente de la interpretación probabilística de la conversación, sino de un conjunto estructurado de datos cuyo origen, estado de validación y utilización en la evaluación puedan determinarse de forma explícita.

### 2.2.2. Resiliencia operativa (DORA)

El Reglamento (UE) 2022/2554, conocido como Digital Operational Resilience Act (DORA), establece requisitos específicos para la gestión del riesgo asociado a las tecnologías de la información. Entre otros aspectos, exige disponer de un marco documentado de gestión del riesgo TIC y de mecanismos destinados a garantizar la resiliencia de los sistemas, la detección y gestión de incidentes y la capacidad de mantener las operaciones ante situaciones adversas.

En un sistema de concesión de crédito basado en un agente conversacional, estas obligaciones adquieren relevancia porque el componente de inteligencia artificial forma parte de un proceso financiero que no puede quedar condicionado exclusivamente por el comportamiento probabilístico del modelo generativo. El modelo puede participar en la comprensión de las respuestas del usuario y en la generación del diálogo, pero no debería constituir por sí mismo la autoridad sobre los datos financieros, las reglas de evaluación o el resultado final de la solicitud.

Esta separación no constituye una prohibición normativa de utilizar modelos generativos ni implica que DORA exija una arquitectura determinista. Se trata de una decisión derivada de la necesidad de controlar el riesgo tecnológico: las funciones críticas del proceso deben permanecer sometidas a mecanismos verificables y controlados, de manera que un error de interpretación, una respuesta generada incorrectamente o una entrada maliciosa no pueda modificar por sí misma los criterios financieros utilizados para evaluar la solicitud.

### 2.2.3. Implicaciones del EU AI Act para el crédito

El Reglamento (UE) 2024/1689, conocido como EU AI Act, establece un régimen específico para determinados sistemas de IA utilizados en la evaluación de la solvencia o en la determinación de la calificación crediticia de personas físicas. Estos sistemas aparecen incluidos entre los usos de alto riesgo recogidos en el Anexo III, con las condiciones y excepciones previstas por el propio Reglamento.

Por ello, el sistema desarrollado en este trabajo no debe limitar el registro a la respuesta final proporcionada al usuario. Debe conservar un estado estructurado de la solicitud y los eventos relevantes del proceso: datos proporcionados, datos validados, consultas a fuentes externas cuando procedan, resultados obtenidos, reglas de negocio aplicadas y resultado de la evaluación. El Reglamento exige capacidades de registro y establece requisitos de trazabilidad que permitan supervisar su funcionamiento.

En conjunto, la regulación específica del crédito desplaza el problema desde la simple interacción conversacional hacia la fiabilidad de la información utilizada

para evaluar la solicitud. El asistente debe ser capaz de recopilar información de forma estructurada, distinguir su procedencia y grado de validación, incorporar fuentes externas cuando corresponda y someter el resultado a reglas financieras explícitas.

## 2.3. Modelado del Dominio y Ontología Financiera

Para que un sistema automatizado cumpla con las exigencias expuestas, el diálogo conversacional debe actuar exclusivamente como la interfaz para poblar un modelo de datos estructurado y determinista. Este modelo, conocido en la ingeniería del conocimiento como ontología, proporciona la arquitectura semántica que permite al sistema "comprender" el entorno bancario.

El modelado del dominio define los conceptos formales que componen el proceso de originación de crédito, sus atributos y las dependencias lógicas entre ellos. Esta estructura establece un estado canónico que actúa como única fuente de verdad durante el análisis de riesgo.

### 2.3.1. Entidades del Ecosistema Bancario

La ontología de un proceso de concesión de crédito requiere la conceptualización de múltiples entidades abstractas:

- **Cliente (Titular):** Representa a la persona física que inicia el proceso. El cliente es el nodo central del que dependen entidades subordinadas como su *Situación Económica* y sus *Obligaciones Financieras*.
- **Solicitud:** Actúa como la entidad contenedora (el expediente). Agrupa al titular, el producto deseado y el estado temporal de la operación (en curso, pendiente de validación, aprobada o denegada), garantizando la integridad transaccional.
- **Producto Financiero:** Define las características exactas del crédito que se está negociando. Sus parámetros definitorios incluyen el importe, la finalidad de la financiación, el plazo de amortización y el tipo de interés aplicable.
- **Situación Económica:** Entidad que modela la capacidad de generación de recursos del titular. Separa conceptualmente los ingresos recurrentes (nóminas, pensiones) de los esporádicos, y categoriza la estabilidad del vínculo laboral (tipo de contrato, antigüedad).
- **Obligaciones Financieras:** Representa el pasivo del cliente. Incluye préstamos vigentes, límites de tarjetas de crédito y otras cargas estructurales que impactan directamente en la liquidez disponible.

- **Avalista (Garante):** Entidad opcional que mitiga el riesgo de la operación. Replica la estructura del *Cliente*, requiriendo una instanciación paralela de su propia situación económica para evaluar su capacidad de respaldo.
- **Evaluación de Riesgo:** Es la entidad resultante. Almacena la resolución algorítmica, los umbrales financieros calculados y la traza de auditoría que justifica legalmente la decisión final.

### 2.3.2. Variables y Atributos de Riesgo

De cada una de las entidades descritas emana un conjunto de variables granulares que nutren los algoritmos de *scoring*. Estas variables se clasifican en cuatro dimensiones fundamentales de riesgo:

1. **Factores Personales y Sociodemográficos:** Atributos como la edad del solicitante y el número de personas a cargo, indispensables para calcular el umbral de subsistencia o mínimo inembargable.
2. **Capacidad de Pago:** Variables que cuantifican la liquidez, como los ingresos netos recurrentes. A nivel de riesgo bancario, un mismo volumen de ingresos recibe ponderaciones radicalmente distintas dependiendo de la categoría laboral (funcionario, trabajador indefinido o autónomo).
3. **Endeudamiento y Cargas:** Cifras necesarias para calcular el ratio de endeudamiento. Se compone de la suma de todas las cuotas mensuales vigentes más una estimación paramétrica de los gastos estructurales.
4. **Garantías:** Atributos de los activos o respaldos patrimoniales que aseguran la operación en caso de insolvencia.

El mayor desafío sistémico en este nivel es la **normalización de la información**. El lenguaje humano es inherentemente difuso. Si un solicitante declara "*gano unos mil quinientos, pero en catorce pagas*", el sistema no puede almacenar una transcripción literal. Debe poseer la capacidad de realizar una normalización semántica, traduciendo la expresión a variables tipadas y estrictas (ej. ingresos anualizados, divididos en doce meses). Sin esta cristalización de conceptos abstractos en tipos de datos estructurados, la evaluación matemática del riesgo resulta inviable.

### 2.3.3. Fuentes de Información

En cumplimiento de los principios de préstamo responsable, la información verbalizada por el solicitante carece de validez ejecutiva hasta ser contrastada. Todo sistema de evaluación debe basar su decisión en información verificada. Para ello, el modelado del dominio contempla la integración con repositorios externos e internos:

- **Central de Información de Riesgos (CIRBE):** Inventario público que acumula el historial crediticio y la exposición total al riesgo de las personas físicas. Su consulta permite aflorar pasivos vigentes que el usuario haya podido omitir involuntaria o deliberadamente.
- **Ficheros de Morosidad (ASNEF, BADEXCUG):** Registros de incumplimientos de obligaciones dinerarias. En el ámbito bancario, una coincidencia positiva en estos registros suele constituir un criterio de exclusión automática, invalidando cualquier capacidad de pago teórica demostrada previamente.
- **Bases de Datos Internas y Agregación Financiera:** Información transaccional histórica del banco o datos extraídos mediante normativas de *Open Banking* (PSD2), que permiten certificar la veracidad de los ingresos y gastos declarados.

El modelo de datos debe mantener un estatus de procedencia estricto, diferenciando siempre si un atributo (como la cuota de un préstamo existente) es un dato "declarado" por el usuario o un dato "verificado" por una de estas fuentes, garantizando la trazabilidad ante futuras auditorías.

### 2.3.4. Gestión de la Incertidumbre

Una diferencia relevante entre la originación de crédito mediante formularios tradicionales y las interfaces conversacionales reside en la forma en que se recopila y actualiza la información. Mientras que los formularios organizan los datos en campos explícitos, las interfaces conversacionales los obtienen progresivamente a lo largo de múltiples turnos. En consecuencia, durante buena parte de la interacción, el sistema debe operar con un expediente cuya información todavía puede estar incompleta, ser ambigua o presentar inconsistencias.

Esta característica exige que la arquitectura gestione el conocimiento parcial y evite que la ausencia de información se interprete como un valor válido o como evidencia suficiente para tomar una decisión. El tratamiento del conocimiento parcial en el dominio crediticio exige distinguir tres situaciones principales:

1. **Información ausente:** Se produce cuando una variable requerida todavía no ha sido proporcionada ni obtenida mediante una fuente válida. Por ejemplo, si se desconocen los gastos mensuales del solicitante y estos son necesarios para evaluar su capacidad de pago, el sistema no debe sustituirlos por estimaciones. En su lugar, debe identificar el dato pendiente y solicitarlo al usuario o activar el mecanismo alternativo de obtención de información previsto por el proceso. La evaluación podrá continuar cuando se cumplan los requisitos de información establecidos para esa etapa.
2. **Información ambigua:** Aparece cuando una declaración no permite determinar de forma suficientemente precisa el valor o la categoría de una variable. Por ejemplo, afirmar que se tiene un "contrato normal" no permite establecer con certeza la modalidad contractual. El sistema debe identificar

la ambigüedad y solicitar una aclaración cuando el dato sea relevante para la evaluación, hasta obtener un valor que pueda normalizarse y validarse conforme al modelo de dominio.

3. **Información inconsistente:** Se produce cuando existen contradicciones entre distintas declaraciones del usuario o discrepancias entre la información declarada y la obtenida de fuentes externas. Por ejemplo, el importe de las obligaciones financieras comunicado durante la conversación puede diferir del que consta en una fuente consultada, como la CIRBE, cuando dicha consulta resulte procedente. El sistema debe registrar la discrepancia y aplicar reglas explícitas para su tratamiento. Cuando la discrepancia sea relevante y no pueda resolverse mediante reglas previamente definidas, deberá solicitarse una aclaración o derivarse el expediente a revisión.

En síntesis, la gestión del conocimiento parcial convierte la evaluación crediticia conversacional en un proceso iterativo. A medida que avanza el diálogo, el sistema actualiza el expediente y comprueba si la información disponible satisface los requisitos de completitud, validez y coherencia establecidos para la siguiente etapa. Cuando estos requisitos se cumplen, el expediente puede pasar al catálogo de reglas de negocio de la entidad. En caso contrario, el sistema debe identificar qué información o validación sigue pendiente y determinar la acción correspondiente, sin confundir la ausencia de datos con el cumplimiento de los criterios de concesión.

## 2.4. Catálogo de Reglas en la Concesión de Crédito

Una vez definido el modelo de información necesario para representar una solicitud de crédito, es necesario establecer cómo dicha información se transforma en una decisión. En una entidad financiera, esta transformación no depende exclusivamente de un modelo estadístico o de la interpretación realizada por el sistema conversacional, sino de un conjunto de políticas, restricciones y criterios de decisión que determinan las condiciones bajo las cuales una operación puede ser concedida, rechazada o sometida a revisión.

Estas reglas constituyen el catálogo de reglas de negocio de la concesión de crédito. Su finalidad es formalizar los criterios aplicables a una solicitud de manera explícita y reproducible, separándolos de la lógica encargada de comprender el lenguaje natural del usuario.

Aunque las reglas concretas dependen de la entidad, del producto financiero y de su política de riesgos, conceptualmente pueden organizarse en diferentes capas de evaluación. En este trabajo se distinguen cuatro categorías:

### 2.4.1. Reglas de Elegibilidad Básica

Las reglas de elegibilidad determinan si una solicitud cumple las condiciones mínimas necesarias para continuar con el proceso de evaluación. Constituyen

una primera capa de filtrado y suelen estar relacionadas con requisitos legales, características del producto o políticas generales de la entidad.

Entre los criterios que pueden formar parte de esta categoría se encuentran la edad del solicitante, las condiciones aplicables al plazo y vencimiento de la operación, el tipo de producto solicitado o determinadas restricciones relativas al destino de la financiación. También pueden existir condiciones relacionadas con la documentación o con la posibilidad de contratar el producto bajo unas determinadas circunstancias.

Su característica principal es que evalúan si la operación puede entrar o continuar en el proceso, antes de analizar en profundidad la capacidad económica y el perfil de riesgo del solicitante. Por tanto, una condición de elegibilidad incumplida puede impedir que tenga sentido continuar con las etapas posteriores de evaluación.

### 2.4.2. Reglas de Viabilidad Financiera

Superados los criterios básicos de elegibilidad, las reglas de viabilidad financiera permiten determinar si la situación económica del solicitante es compatible con las obligaciones derivadas de la nueva operación. El objetivo no es establecer todavía el riesgo global del cliente, sino evaluar su capacidad económica para asumir la financiación solicitada.

Estas reglas pueden combinar diferentes variables relacionadas con los ingresos, gastos, obligaciones financieras existentes, importe solicitado y condiciones de la nueva operación. Entre las métricas habituales se encuentran los ratios de endeudamiento, la relación entre ingresos y obligaciones financieras y el importe de renta disponible después de atender los compromisos previstos.

Una regla de este tipo puede, por ejemplo, comprobar que la cuota resultante de la nueva financiación, junto con las obligaciones financieras existentes, se mantiene dentro de los límites establecidos por la política de riesgos de la entidad. De forma complementaria, pueden establecerse umbrales mínimos de renta disponible que permitan evitar que el pago de las obligaciones financieras comprometa excesivamente la capacidad económica del solicitante.

Los valores concretos de estos límites no son universales, sino que dependen del producto, de la entidad y de su política de riesgos.

### 2.4.3. Reglas de Riesgo

La capacidad económica constituye una dimensión necesaria, pero no suficiente, para determinar la conveniencia de conceder una operación. Las reglas de riesgo incorporan información adicional relacionada con el historial de cumplimiento, la exposición crediticia y otras circunstancias que pueden modificar la valoración de una solicitud incluso cuando su capacidad de pago aparente ser suficiente.

Dentro de esta categoría pueden encontrarse reglas relacionadas con la existencia de obligaciones impagadas o incidencias registradas en fuentes de información crediticia. También pueden establecerse reglas para tratar situaciones en las que

exista una discrepancia significativa entre la información declarada y la información obtenida mediante fuentes verificadas. Estas discrepancias no implican necesariamente una intención fraudulenta, ya que pueden deberse a errores de comunicación, información desactualizada o diferencias entre las definiciones utilizadas por distintas fuentes. Por este motivo, su tratamiento puede consistir en solicitar aclaraciones, requerir documentación adicional o derivar el expediente a revisión, en función de la política definida por la entidad.

La principal diferencia respecto a las reglas de viabilidad es, por tanto, que estas reglas no se limitan a responder si el cliente *puede* asumir matemáticamente la operación, sino que incorporan otros elementos relevantes para determinar si el nivel de riesgo asociado a dicha operación resulta aceptable para la entidad.

### 2.4.4. Reglas de Decisión y Resultados

Las reglas anteriores producen resultados parciales que deben ser consolidados para determinar el estado final de la solicitud. Esta última categoría establece cómo se combinan las diferentes evaluaciones y qué resultado corresponde a cada situación.

De forma general, pueden distinguirse tres resultados principales:

1. **Aprobación:** se produce cuando la solicitud satisface los criterios de elegibilidad y los resultados de las evaluaciones de capacidad y riesgo se encuentran dentro de los límites establecidos para la operación. En determinados productos y bajo las condiciones definidas por la entidad, este resultado puede producirse de forma automatizada.

2. **Denegación:** se produce cuando se incumple una condición que, de acuerdo con la política de la entidad, impide la concesión de la operación. El resultado debe estar asociado a los criterios que han conducido a dicha decisión, permitiendo reconstruir qué condiciones del expediente fueron determinantes.

3. **Revisión o intervención humana:** determinados expedientes pueden no encajar de forma inequívoca en las condiciones de aprobación o denegación automática. Esto puede ocurrir ante discrepancias relevantes, situaciones próximas a determinados umbrales o circunstancias que requieren una valoración adicional.

Esta separación permite distinguir entre las reglas que evalúan una determinada dimensión del expediente y las reglas que determinan cómo se consolida el resultado de dichas evaluaciones. El catálogo no constituye, por tanto, una única regla de concesión, sino un conjunto estructurado de criterios que, aplicados sobre un expediente suficientemente completo y validado, permiten obtener un resultado reproducible.

Desde la perspectiva de un sistema conversacional, esta separación resulta especialmente relevante. El modelo de lenguaje no debe determinar por sí mismo qué condiciones financieras son aceptables ni modificar los criterios de decisión en función de la conversación. Su función consiste en facilitar la adquisición y

estructuración de información, mientras que la aplicación de las reglas de negocio debe permanecer bajo mecanismos explícitos y controlables.

## 2.5. Perfiles de Usuario

La naturaleza conversacional introduce una alta variabilidad en la interacción, ya que puede recibir información completa y estructurada, datos incompletos o contradictorios, o incluso entradas diseñadas deliberadamente para alterar el comportamiento del sistema.

Por este motivo, el diseño del asistente no debe considerar únicamente el caso nominal de un usuario que proporciona correctamente toda la información solicitada. Es necesario contemplar diferentes perfiles de interacción que permitan identificar las situaciones que el sistema deberá gestionar durante la originación. Estos perfiles no representan necesariamente categorías de clientes reales, sino escenarios de interacción utilizados para analizar los requisitos funcionales y de seguridad del sistema.

### 2.5.1. Cliente con Información Completa

Representa el escenario nominal del proceso. El usuario conoce el producto que desea contratar, responde de forma coherente a las preguntas planteadas y proporciona la información necesaria para completar el expediente.

En este escenario, los datos relevantes pueden obtenerse progresivamente sin que aparezcan contradicciones significativas ni ambigüedades que requieran ciclos adicionales de aclaración.

Este escenario constituye el caso de referencia del sistema, pero no debe interpretarse como una garantía de aprobación. Que la interacción sea completa y fluida únicamente significa que no existen incidencias conversacionales que impidan realizar la evaluación. El resultado final seguirá dependiendo de los criterios de elegibilidad, capacidad financiera y riesgo definidos para la operación.

### 2.5.2. Cliente con Información No Declarada

En este escenario, la información proporcionada durante la conversación puede presentar inicialmente un perfil aparentemente compatible con la operación solicitada. Sin embargo, durante las fases posteriores de validación pueden aparecer circunstancias que no estaban presentes en la información declarada inicialmente.

Estas circunstancias pueden incluir obligaciones financieras no mencionadas durante la conversación, exposición crediticia indirecta o información procedente de fuentes externas que modifique la valoración de la solicitud. Por ejemplo, el solicitante puede declarar unos determinados niveles de endeudamiento mientras una fuente de información crediticia, cuando su consulta resulte procedente, revela obligaciones adicionales.

El escenario pone de manifiesto que la información conversacional no constituye necesariamente una representación completa del expediente. El sistema debe ser

capaz de incorporar nueva información a medida que avanza el proceso y reevaluar el estado de la solicitud cuando aparecen datos que modifican las condiciones previamente consideradas.

La relevancia de este perfil no reside en asumir que la omisión de información sea deliberada. Una discrepancia puede deberse a desconocimiento, errores del usuario, diferencias temporales entre fuentes o una declaración intencionadamente incompleta. El sistema debe, por tanto, detectar y gestionar la discrepancia sin atribuir automáticamente una intención al solicitante.

### 2.5.3. Usuario Ambiguo o Errático

Este perfil representa una de las situaciones características de la interacción conversacional: el usuario no proporciona necesariamente la información requerida de manera ordenada ni en una única respuesta. Puede aportar datos fragmentados a lo largo de varios turnos, corregir respuestas anteriores, cambiar las condiciones de la operación o introducir información que no resulta relevante para la evaluación.

También pueden producirse contradicciones entre diferentes momentos de la conversación. Por ejemplo, el usuario puede declarar inicialmente unos ingresos y posteriormente proporcionar una cifra diferente, o modificar el importe solicitado después de que ya se hayan realizado determinados cálculos.

Este comportamiento impide tratar la conversación como una secuencia lineal de preguntas y respuestas independientes. El sistema debe mantener una representación persistente del estado de la solicitud y determinar en cada momento qué información se encuentra disponible, cuál ha sido corregida y qué datos continúan pendientes de validación.

### 2.5.4. Usuario Adversarial

El último escenario contempla interacciones en las que el usuario intenta deliberadamente modificar el comportamiento previsto del sistema o acceder a un resultado que no le correspondería según las reglas establecidas. A diferencia de los perfiles anteriores, el problema no procede de una interacción imperfecta, sino de una posible intención de manipulación.

Estas amenazas pueden adoptar diferentes formas. En el plano social, un usuario podría intentar utilizar presión emocional, información falsa o técnicas de ingeniería social para conseguir que el sistema ignore determinadas restricciones o interprete favorablemente una situación de riesgo. En el plano técnico, pueden aparecer intentos de *prompt injection*, instrucciones introducidas dentro de la conversación para alterar las instrucciones operativas del sistema, modificar su comportamiento o conseguir que revele información que no debería proporcionar.

El sistema debe, por tanto, mantener una separación clara entre la interpretación de las entradas del usuario y la aplicación de las reglas que determinan la evaluación crediticia. La capacidad del modelo para comprender lenguaje natural no debe implicar que pueda modificar los criterios de decisión ni conceder al usuario control sobre el estado interno de la solicitud.

En conjunto, estos cuatro escenarios muestran que la dificultad del proceso conversacional no reside únicamente en obtener los datos necesarios para evaluar una solicitud, sino en mantener una representación coherente y controlada de dicha solicitud ante diferentes patrones de interacción. Desde el flujo nominal hasta las situaciones de inconsistencia o manipulación, el sistema debe preservar la integridad del proceso de evaluación independientemente de la forma en que el usuario interactúe con él.

## 2.6. Requisitos de Diseño Derivados del Dominio

El análisis del dominio financiero, sus restricciones regulatorias y los diferentes escenarios de interacción identificados en las secciones anteriores permiten derivar un conjunto de requisitos arquitectónicos específicos. El objetivo es determinar qué propiedades debe satisfacer cualquier solución que pretenda integrar un agente conversacional en un proceso de concesión de crédito sin delegar en el modelo generativo el control de la lógica financiera.

Estos requisitos se centran en tres aspectos fundamentales:

### 2.6.1. Necesidad de un Estado Canónico de la Solicitud

Como se ha observado al analizar los escenarios de interacción, la comunicación mediante lenguaje natural es incremental, fragmentada y susceptible de rectificaciones. Una misma variable puede aparecer en diferentes momentos de la conversación, ser modificada posteriormente o requerir validación mediante una fuente externa. Por este motivo, el historial de mensajes no constituye por sí mismo una representación adecuada del estado financiero de la solicitud.

Se requiere una representación estructurada y persistente del estado de la solicitud que sea independiente del historial conversacional. En este trabajo, esta representación se denomina **Information State Update (ISU)** y se construye a partir del modelo de dominio definido anteriormente.

El ISU actúa como la fuente de verdad del proceso de evaluación, en él se almacenan los datos relevantes de la solicitud. De este modo, es posible distinguir entre un dato que todavía no ha sido proporcionado, uno declarado por el usuario y uno que ha sido posteriormente verificado mediante una fuente adicional.

Esto permite aplicar los mecanismos de gestión del conocimiento parcial descritos anteriormente y evita que la continuidad del proceso dependa exclusivamente de la capacidad del modelo de lenguaje para recuperar información del historial conversacional. Asimismo, permite que los diferentes componentes del sistema trabajen sobre una representación común y estructurada de la solicitud.

### 2.6.2. Separación entre Comprensión y Lógica

La necesidad de mantener bajo control las reglas que determinan la evaluación crediticia conduce a una segunda propiedad fundamental: la separación entre el procesamiento del lenguaje natural y la lógica de negocio.

El modelo generativo puede desempeñar funciones de comprensión y generación lingüística, como interpretar las respuestas del usuario, extraer información relevante o formular preguntas de seguimiento. Esta participación no supone que el modelo deba quedar excluido del tratamiento de los datos estructurados. Por ejemplo, ante una expresión como "quiero solicitar 1000 euros", el modelo puede identificar el importe solicitado y producir una representación estructurada como `importe_solicitado = 1000`. El dato puede incorporarse al estado de la solicitud como información declarada por el usuario, manteniendo su procedencia y naturaleza.

La separación de responsabilidades se establece en un nivel diferente: el modelo generativo no debe constituir la autoridad sobre la utilización financiera de esos datos. A partir del importe extraído, por ejemplo, no corresponde al modelo determinar si la cantidad es viable, calcular por sí mismo la capacidad de pago, consultar arbitrariamente una fuente de riesgo o decidir el resultado de la solicitud. Estas operaciones deben realizarse mediante componentes explícitos del sistema, de acuerdo con las reglas y procedimientos definidos para el dominio.

Las reglas de negocio, los cálculos financieros y las condiciones de decisión deben permanecer, por tanto, en componentes cuya ejecución sea reproducible. Esto permite que un error de interpretación, una respuesta incorrecta o una instrucción de manipulación del lenguaje no pueda modificar directamente los criterios de concesión ni determinar por sí misma el resultado financiero. En particular, un intento de *prompt injection* no debería poder alterar una regla de negocio, modificar arbitrariamente el estado de la solicitud o convertir una instrucción proporcionada por el usuario en una decisión de concesión.

La frontera relevante no consiste, por tanto, en impedir que el modelo generativo participe en el tratamiento de la información, sino en delimitar sus responsabilidades. De este modo, la interpretación probabilística del lenguaje no constituye la autoridad final sobre la evaluación crediticia.

### 2.6.3. Trazabilidad y Auditabilidad de las Decisiones

El último requisito deriva de la necesidad de poder reconstruir el proceso mediante el cual una solicitud alcanza un determinado resultado. En un sistema de concesión de crédito, no resulta suficiente conservar únicamente la respuesta final mostrada al usuario. También debe ser posible determinar qué información estaba disponible, qué validaciones se realizaron y qué reglas intervinieron en la evaluación.

Por ello, el sistema debe disponer de mecanismos de registro que permitan conservar los eventos relevantes del proceso y protegerlos frente a modificaciones no autorizadas. Este registro debe complementar al estado actual de la solicitud, manteniendo el historial necesario para reconstruir la evolución del expediente.

Para cada decisión relevante, la traza debería permitir relacionar, al menos, tres dimensiones:

1. **Datos utilizados:** valores empleados durante la evaluación, junto con su procedencia y estado de validación.

2. **Reglas y operaciones aplicadas:** criterios de negocio, cálculos y validaciones que intervinieron en la evaluación, incluyendo la versión de las reglas cuando resulte necesario para reproducir el resultado.

3. **Resultado y causa de la decisión:** estado alcanzado por la solicitud y condiciones que determinaron dicho resultado.

Esta estructura permite que la explicación de una decisión se construya a partir de información efectivamente registrada durante la ejecución, en lugar de solicitar al modelo generativo una justificación retrospectiva de un resultado cuyo proceso interno no ha quedado documentado. La trazabilidad se convierte así en una propiedad del proceso de decisión y no únicamente de la respuesta conversacional.

En conjunto, los tres requisitos definidos delimitan las propiedades fundamentales que debe satisfacer la arquitectura propuesta. Estos requisitos permiten trasladar las características específicas del dominio crediticio a decisiones concretas de diseño, cuya implementación se desarrolla en los capítulos posteriores.

# Capítulo 3

# Implementación de la arquitectura propuesta

El capítulo anterior definió los requisitos que debe satisfacer el sistema: una representación estructurada del estado de la solicitud independiente del historial conversacional, separación entre la interacción en lenguaje natural y la lógica de evaluación crediticia, y un mecanismo de trazabilidad que permita auditar cada decisión. Este capítulo describe cómo esos tres requisitos se materializan en un prototipo funcional.

## 3.1. Visión general de la arquitectura

La implementación concreta el paradigma *Symbolic[Neuro]* mediante una arquitectura en la que las capacidades de procesamiento del lenguaje se mantienen separadas de la lógica de decisión del proceso de evaluación crediticia. El sistema recibe la solicitud del usuario en lenguaje natural, la transforma en una representación estructurada y somete dicha representación a una evaluación basada en reglas explícitas.

El núcleo simbólico de la arquitectura está constituido por el motor de reglas. A partir del estado estructurado de la solicitud, este componente determina el resultado de la evaluación y establece la siguiente acción que debe realizarse. De esta forma, el control funcional del proceso reside en las reglas de negocio y no en el modelo de lenguaje.

La ejecución técnica de las acciones determinadas por el motor se realiza mediante el backend de la aplicación. Este componente actúa como intermediario entre los distintos servicios: recibe la información de la interacción, mantiene el estado necesario durante el procesamiento, ejecuta las acciones indicadas por el motor de reglas y vuelve a someter el estado actualizado a evaluación cuando corresponde.

El modelo de lenguaje interviene únicamente en las tareas relacionadas con el lenguaje natural. La información obtenida de la conversación se incorpora al estado estructurado de la solicitud y, una vez finalizada la evaluación, el resultado obtenido se utiliza para generar la respuesta dirigida al usuario. La decisión

crediticia se mantiene, por tanto, fuera de la generación textual del modelo.

Además, el sistema incorpora mecanismos de persistencia que permiten conservar el estado utilizado durante la evaluación y registrar las operaciones realizadas a lo largo del proceso. Por otro lado, cuando la evaluación requiere información procedente de fuentes externas, el backend puede ejecutar la acción correspondiente sobre un servicio simulado que representa dichas fuentes.

### 3.1.1. Organización de los componentes

Para el despliegue de la arquitectura descrita, hemos elegido varios servicios y tecnologías que se comunican entre sí durante el procesamiento de una solicitud. La tabla siguiente recoge la correspondencia entre los componentes y las tecnologías utilizadas en el prototipo.

| **Componente** | **Tecnología** | **Materialización en el prototipo** |
|---|---|---|
| Backend y coordinador técnico | Python 3.11 / FastAPI | Expone la API de la aplicación, mantiene la comunicación entre servicios y ejecuta técnicamente las acciones determinadas por el motor de reglas. |
| Procesamiento del lenguaje | Mistral AI API | Proporciona las funciones de comprensión del lenguaje natural y generación de respuestas mediante llamadas a la API del modelo. |
| Motor de reglas | Java 17 / Spring Boot / Drools KIE | Ejecuta las reglas de negocio sobre el estado estructurado de la solicitud y devuelve el resultado de la evaluación y, cuando corresponde, la acción que debe ejecutarse a continuación. |
| Persistencia | MariaDB 11.2 | Almacena los datos utilizados por el prototipo y las trazas generadas durante el procesamiento de las solicitudes. |
| Servicio de solvencia | Python | Simula el acceso a fuentes externas de información de riesgo, proporcionando los datos necesarios para determinadas reevaluaciones. |

Tabla 3.1: Componentes tecnológicos de la arquitectura implementada.

La implementación se despliega como un conjunto de servicios independientes, lo que permite mantener separadas las responsabilidades definidas en la arquitectura. Los detalles de configuración, comunicación y ejecución de cada componente se desarrollan en las secciones posteriores.

### 3.1.2. Paradigma neuro-simbólico

La distribución anterior se corresponde con el paradigma **Symbolic[Neuro]**. **Drools** es el único componente que determina si una solicitud debe aprobarse,

denegarse o derivarse a supervisión humana. **Mistral** desempeña exclusivamente funciones lingüísticas: transforma el texto del usuario en datos estructurados (NLU) y convierte el resultado de Drools en lenguaje natural (NLG). En ninguno de los dos casos puede modificar el estado canónico de la solicitud. **FastAPI** coordina técnicamente la secuencia de acciones, sin implementar lógica de negocio.

Esta separación tiene una consecuencia de seguridad relevante: un error de interpretación de Mistral o un intento de *prompt injection* puede afectar a los campos extraídos por NLU, pero la decisión crediticia se produce sobre el estado estructurado mediante reglas deterministas e independientes del modelo de lenguaje.

### 3.1.3. Flujo general de una solicitud

A partir de esta distribución de responsabilidades, una solicitud atraviesa una secuencia de etapas. El detalle de la implementación de cada componente se desarrolla en las secciones posteriores, en esta sección se presenta únicamente el flujo global.

1. **Recepción y comprensión del mensaje**: el usuario proporciona una petición en lenguaje natural. El backend la envía al componente de procesamiento del lenguaje, que extrae la información relevante y la representa mediante una estructura de datos preestablecida.

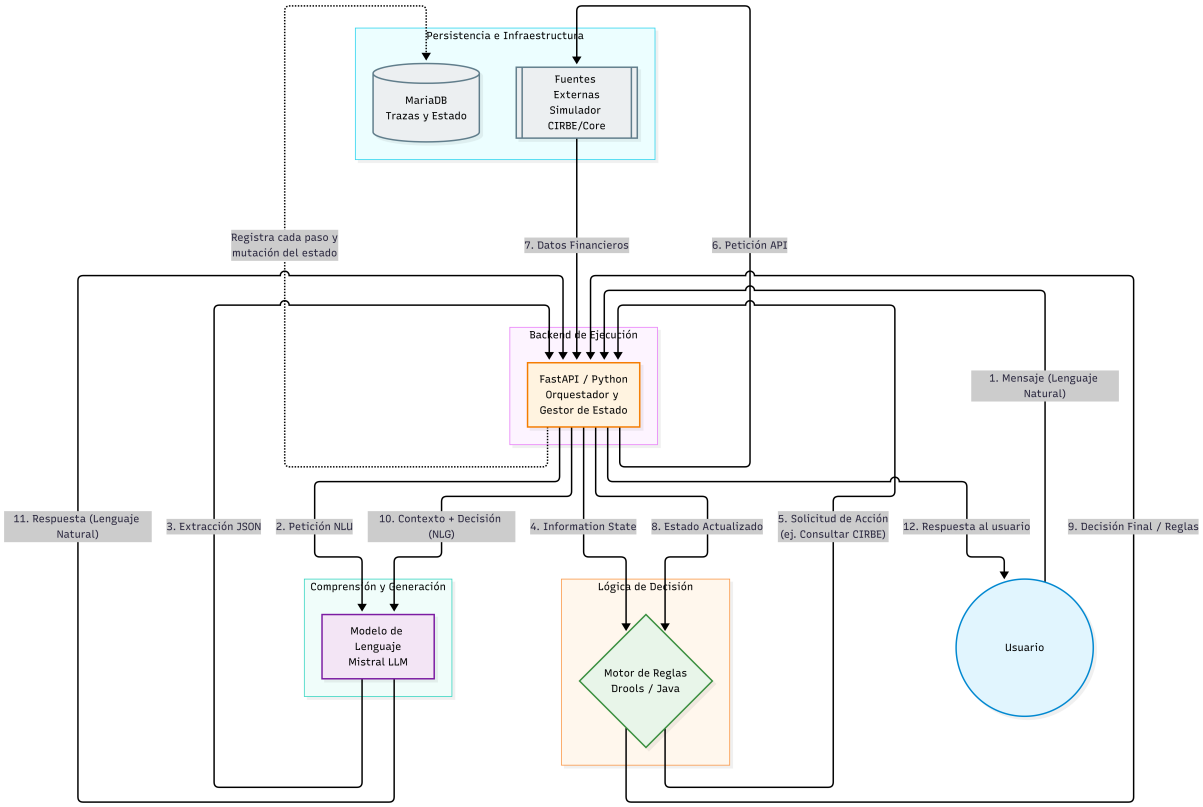
2. **Incorporación de información bancaria**: el sistema recupera, cuando corresponde, los datos disponibles en el entorno bancario simulado y los incorpora a la información de la solicitud.

3. **Evaluación mediante reglas**: el backend envía el estado estructurado al motor de reglas. Este aplica las condiciones definidas para el prototipo y devuelve las reglas aplicadas junto con el resultado de la evaluación o la información necesaria para determinar el siguiente paso del proceso.

4. **Consulta de información externa**: cuando la evaluación determina que es necesaria información adicional, el backend solicita los datos correspondientes al servicio simulado de solvencia. La nueva información se incorpora al estado y se realiza una nueva evaluación.

5. **Generación de la respuesta**: una vez obtenido el resultado final de la evaluación, el sistema proporciona la información resultante al componente de procesamiento del lenguaje para generar la respuesta que se presenta al usuario.

La Figura muestra de manera clara el recorrido general de una solicitud, desde su recepción hasta la obtención del resultado y la generación de la respuesta.




Las distintas etapas del proceso se registran en la base de datos junto con la información de estado necesaria para mantener la continuidad de la solicitud y proporcionar trazabilidad sobre su procesamiento.

## 3.2. Decisiones tecnológicas

### 3.2.1. Por qué Drools como motor de reglas

La elección de un motor de reglas basado en lógica de producción, frente a codificar la lógica en Python o delegarla en el modelo de lenguaje, responde a tres propiedades fundamentales del dominio crediticio: determinismo, auditabilidad y separación entre política y código.

Frente a alternativas orientadas a la definición determinista de flujos de ejecución, como Camunda, Drools permite trabajar con un estado de hechos sobre el que las reglas pueden evaluarse de forma incremental. El motor mantiene este estado en memoria y, cuando se incorporan nuevos datos o se modifica alguno de los existentes, puede volver a evaluar las reglas que dependen de ellos. Esta característica, habitualmente denominada *stateful*, permite que una decisión no tenga que definirse como una secuencia fija de pasos, sino que pueda obtenerse a partir de las condiciones que se cumplen en cada momento.

En el contexto del prototipo, esta capacidad permite que la incorporación progresiva de información al *InformationState* desencadene nuevas evaluaciones. Por ejemplo, la disponibilidad de los datos económicos del cliente puede permitir

calcular determinados indicadores y, a partir de estos valores, activar reglas adicionales de evaluación de solvencia.

Asimismo, Drools proporciona el lenguaje DRL (*Drools Rule Language*), diseñado para expresar reglas de negocio mediante una separación entre las condiciones que deben cumplirse y las acciones que se ejecutan cuando estas se satisfacen. Por ejemplo, una condición como la superación del umbral de endeudamiento puede expresarse directamente como una regla que modifica el estado de la solicitud y establece el resultado correspondiente. Esta separación permite mantener las reglas de decisión crediticia fuera de la lógica general de la aplicación y facilita su revisión y mantenimiento. Esto resulta especialmente adecuado para un sistema en el que las decisiones de negocio deben poder revisarse y modificarse sin alterar el resto de los componentes.

Otra característica relevante para el prototipo es el mecanismo de priorización de reglas mediante el atributo *salience*. Cuando varias reglas son aplicables simultáneamente, *salience* permite establecer explícitamente cuál debe evaluarse primero. En este caso, las reglas que representan condiciones de denegación inmediata se configuran con una prioridad superior a las reglas que conducen a estados posteriores, como la aprobación. Por ejemplo, una solicitud que incumple el requisito de edad mínima debe ser tratada antes de que pueda activarse una regla de aprobación estándar.

### 3.2.2. Por qué Mistral como modelo de lenguaje

Para la implementación del componente de procesamiento del lenguaje se evaluaron modelos de lenguaje de distintas familias, entre ellas Mistral y Llama. La selección de Mistral estuvo condicionada principalmente por su adecuación a las tareas requeridas por el prototipo y por la facilidad de integración mediante su API. Al tratarse de un prototipo académico, también se tuvo en consideración la disponibilidad de modelos y servicios con un coste reducido para facilitar las pruebas y el desarrollo experimental.

La principal razón técnica para seleccionar Mistral fue su integración con mecanismos de salida estructurada. En la implementación, esta funcionalidad se utiliza mediante el SDK de Mistral y el método `chat.parse()`, al que se proporciona un esquema Pydantic mediante el parámetro `response_format`. De esta forma, la respuesta del modelo se obtiene directamente en una estructura que puede ser validada y utilizada por el backend, reduciendo la necesidad de realizar procesamiento adicional sobre texto libre.

Finalmente, la integración se ha diseñado de forma desacoplada del modelo concreto. La identificación del modelo utilizado se realiza mediante la variable de entorno `MISTRAL_MODEL`, de modo que su sustitución o actualización puede realizarse sin modificar la lógica principal del backend. Esta separación permite mantener estable la arquitectura, aunque se sustituya el modelo de lenguaje empleado en futuras versiones.

### 3.2.3. Por qué Pydantic para el esquema de datos

Pydantic actúa como capa de validación tipada en todos los puntos del *pipeline*. Primero, el objeto `InformationState` es la fuente de verdad del contrato de datos: cualquier campo que no exista en el modelo Pydantic no puede propagarse al motor de reglas por diseño. Segundo, la integración con el SDK de Mistral es directa: al ejecutar `chat.parse(response_format=EsquemaNLU)`, Pydantic se emplea como la especificación estricta del JSON esperado.

### 3.2.4. Por qué FastAPI como backend

El sistema requiere un componente backend que actúe como punto de integración entre el modelo de lenguaje, el motor de reglas, la capa de persistencia y los servicios externos utilizados durante la evaluación. En lugar de delegar esta coordinación en un *framework* específico de agentes, se optó por implementar un flujo explícito en Python, de forma que las operaciones realizadas entre los distintos componentes quedasen definidas directamente en la aplicación.

FastAPI se utilizó como base para este backend debido a su soporte para la definición de APIs HTTP y su integración con Pydantic para la validación de los datos intercambiados entre los servicios. Estas características permiten definir interfaces estructuradas entre los distintos componentes y mantener separadas las responsabilidades de comunicación, gestión del estado y ejecución de las operaciones.

Finalmente, FastAPI proporciona documentación interactiva de las interfaces HTTP mediante OpenAPI, accesible durante el desarrollo a través de `/docs`. Esta funcionalidad facilitó la comprobación y depuración de las interfaces entre los servicios que forman parte del prototipo.

## 3.3. Arquitectura e infraestructura del sistema

Esta sección detalla la materialización técnica del prototipo. El sistema se ha construido bajo una arquitectura orientada a microservicios, lo que garantiza un acoplamiento débil entre las distintas tecnologías empleadas (Python, Java y SQL) y facilita la escalabilidad independiente de la capa de orquestación frente a la capa de inferencia simbólica.

### 3.3.1. Arquitectura física y despliegue con Docker Compose

El entorno de ejecución se orquesta mediante Docker Compose, encapsulando los servicios en contenedores aislados que se comunican a través de una red virtual privada interna denominada *banca_neurosimbolico_net*. Esta topología aísla los componentes del exterior, exponiendo únicamente los puertos necesarios. El despliegue consta de tres contenedores principales:

- **Capa de Persistencia (*banca_mariadb*):** Basado en la imagen oficial de MariaDB 11.2. Se inicializa de forma automatizada ejecutando el *script* `database/init.sql` durante el primer arranque. Los datos persisten en el volumen local *banca_neurosimbolico_db*, garantizando la inmutabilidad de los datos y, por tanto, de la auditoría ante reinicios del contenedor. Incorpora un *healthcheck* que verifica la disponibilidad del motor antes de permitir el arranque de los servicios dependientes.

- **Motor Simbólico (*banca_drools*):** Implementado como un microservicio Spring Boot. Su construcción emplea un enfoque *multi-stage build*: una primera fase con Maven compila el artefacto JAR y sus dependencias (ecosistema KIE de Drools), y una segunda fase con Eclipse Temurin JRE 17 lo ejecuta. Para optimizar el rendimiento del motor de reglas, la configuración del servicio limita la memoria disponible para la máquina virtual de Java a un máximo de 512 MB. Su disponibilidad se monitoriza mediante el *endpoint* `/actuator/health` de Spring Boot Actuator.

- **Orquestador Backend (*banca_backend*):** Basado en la imagen oficial de Python 3.11. Actúa como el único punto de entrada del sistema, sirviendo además los archivos estáticos del *frontend* bancario desde el directorio `/static/`. Para prevenir condiciones de carrera durante el despliegue, el arranque de este contenedor está supeditado a los *healthchecks* exitosos de MariaDB y Drools.

### 3.3.2. Comunicación entre servicios e interfaces

La separación tecnológica de los servicios exige protocolos de comunicación estandarizados. El diseño de las interfaces y el flujo de datos se ha resuelto de la siguiente manera:

- **Backend (FastAPI) ↔ motor de reglas (Drools):** La comunicación se realiza mediante peticiones HTTP POST síncronas utilizando la biblioteca *httpx* en Python. FastAPI estructura los datos que envía mediante el modelo Pydantic `DroolsRequest`. Estos datos se serializan en formato JSON y se envían al microservicio Java, donde Spring Boot los deserializa en un objeto de tipo `SolicitudEvaluacion`. El servicio utiliza los datos recibidos para alimentar una sesión de Drools, ejecutar las reglas mediante `fireAllRules()` y devolver el resultado de la evaluación a FastAPI.

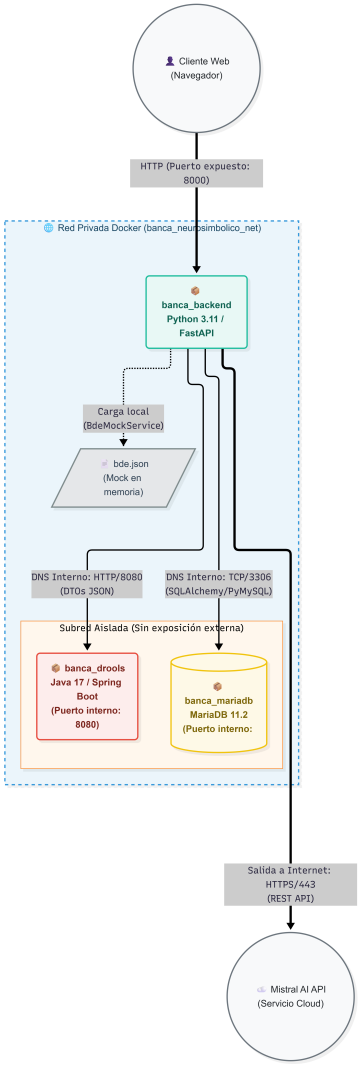
- **Orquestador (FastAPI) ↔ Modelo de Lenguaje (Mistral):** El sistema se comunica con la API de Mistral mediante su SDK oficial utilizando una api key declarada en el `.env`. Para las tareas de extracción (NLU), se invoca el método `chat.parse()`. Para la tarea de generación (NLG), se utiliza `chat.complete()`.

- **Orquestador (FastAPI) ↔ Persistencia (MariaDB):** El acceso a la base de datos se realiza mediante SQLAlchemy, una biblioteca de Python que permite interactuar con las tablas relacionales mediante objetos y operaciones

de Python sin necesidad de escribir manualmente todas las consultas SQL, siguiendo el enfoque conocido como ORM (*Object-Relational Mapper*). Para establecer la conexión con MariaDB se utiliza el dialecto `mysql+pymysql`, que especifica el sistema gestor de bases de datos y el controlador empleado.

- **Orquestador (FastAPI) ↔ Mock de Solvencia (Datos Externos):** En lugar de introducir latencia de red adicional, la simulación de servicios externos (CIRBE/ASNEF) se ha integrado en la capa de servicios de FastAPI. La clase `BdeMockService` carga el archivo `bde.json` en memoria durante la inicialización y sirve los datos financieros mediante búsquedas en diccionarios locales (por DNI), simulando el comportamiento de un sistema *core* bancario.

La figura siguiente sintetiza visualmente este esquema de interacciones:




### 3.3.3. Estructura y organización del proyecto

El código del prototipo se ha organizado siguiendo patrones de diseño de separación de responsabilidades, dividiendo el repositorio en los siguientes bloques:

- **`backend/`**: Contiene la lógica del orquestador en FastAPI. El subdirectorio `app/` incluye `main.py` y agrupa la lógica de negocio en capas: `services/` para el orquestador principal y servicios auxiliares, `clients/` para las integraciones con Mistral y Drools, `schemas/` para los modelos de datos (Pydantic), `models/` para el mapeo ORM (SQLAlchemy), y `routers/` para la exposición de los *endpoints*. Adicionalmente, incluye el *frontend* en `static/`, una práctica no muy profesional, pero suficiente para el foco principal del tfm.
- **`drools-app/`**: Contiene el microservicio en Spring Boot. Las reglas de negocio DRL, elemento central de la lógica simbólica, residen en `src/main/resources/rules/prestamos.drl`. Los modelos de datos Java se agrupan en el paquete `model/`.
- **`database/`**: Agrupa el *script* de inicialización (`init.sql`) que definen el esquema DDL y los datos de prueba poblados durante el despliegue.
- **`tests/`**: Contiene la batería de pruebas automatizadas. Comentaremos esto en el siguiente capítulo más en profundidad.

## 3.4. El *Information State Update* (ISU)

El `InformationState` constituye el contrato de datos central del sistema, implementado como un modelo Pydantic en `schemas/state.py`. Este objeto actúa como la única fuente de verdad durante la evaluación. Se compone de cinco bloques de información, cada uno con un ciclo de poblamiento y procedencia definidos. Además, para garantizar la integridad de la solicitud, la arquitectura restringe cómo y cuándo muta la información dentro de cada bloque del esquema:

- **`datos_usuario`:** Contiene las entidades extraídas por el modelo de lenguaje. En particular, extrae la intención principal, la cantidad solicitada, el plazo en meses, el propósito, los gastos fijos mensuales y la deuda declarada. Incluye también un campo para recoger dudas o consultas del cliente que no encajen en los campos estructurados anteriores. Se actualiza turno a turno mediante un proceso de *fusión acumulativa*: si el componente NLU extrae un valor válido, este sobrescribe el campo; si no extrae información (devuelve `null`), se preserva intacto el valor del turno anterior.
- **`datos_core_bancario`:** Representa el perfil financiero estático del cliente dentro del banco. De aquí se obtienen los siguientes datos: identificador, nombre, DNI, edad, ingresos netos mensuales, antigüedad en meses del cliente, la deuda interna dentro del banco y si el usuario es avalista de alguna empresa. Para optimizar el rendimiento, esta información se recupera de la base de datos (`clientes`) una única vez durante el primer turno. En interacciones posteriores, el orquestador recupera esta información directamente desde el último estado persistido.

- **`datos_externos_bde`:** Alberga la información de solvencia simulada (CIRBE y ASNEF). Su poblamiento es condicional: el campo `consultado` se inicializa en `false` y, si es necesario, Drools dispara la regla específica para su consulta. Una vez realizada la llamada al servicio externo y obtenidos los datos de deuda y morosidad, la bandera pasa a `true`, bloqueando ejecuciones redundantes en turnos posteriores. Contiene los siguientes datos: si tiene impagos activos, el importe de morosidad, la deuda mensual y total, y si la empresa avalada está en concurso.
- **`evaluacion_drools`:** Almacena el resultado emitido por el motor simbólico tras cada iteración. Incluye el estado de la solicitud, las reglas disparadas, la siguiente acción requerida, el dato faltante, el artículo normativo, el texto legal aplicable y el tono recomendado para la generación de la respuesta.
- **`calculos_derivados`:** Agrupa las métricas financieras necesarias para evaluar la viabilidad de la operación.

Estos cálculos derivados son realizados por el microservicio Java (`DroolsService`, mediante el método `calcularDerivados()`) antes de insertar los *facts* en la memoria de trabajo de Drools. Su función es transformar los datos disponibles en indicadores financieros que puedan utilizar las reglas de negocio. Las fórmulas implementadas son:

$$\text{cuota_estimada} = \frac{\text{cantidad_solicitada}}{\text{plazo_meses}} \tag{3.1}$$

$$\begin{aligned}\text{total_compromisos} = {} & \text{cuota_estimada} + \text{gastos_fijos_mens} \\ & + \text{deuda_interna_mens} + \text{deuda_cirbe_mens}\end{aligned} \tag{3.2}$$

$$\text{ratio_endeudamiento} = \frac{\text{total_compromisos}}{\text{ingresos_netos_mens}} \tag{3.3}$$

$$\text{remanente_subsistencia} = \text{ingresos_netos_mens} - \text{total_compromisos} \tag{3.4}$$

$$\text{edad_al_vencimiento} = \text{edad} + \left\lfloor \frac{\text{plazo_meses}}{12} \right\rfloor \tag{3.5}$$

La implementación adopta una estrategia de cálculo parcial que permite evaluar la solicitud con la información disponible en cada momento. La cuota estimada solo se calcula si se conocen la cantidad solicitada y el plazo, y este último es distinto de cero. A partir de esa cuota, el total de compromisos se calcula sumando los gastos fijos y las deudas mensuales cuyos valores están disponibles. Los conceptos todavía desconocidos no se incorporan al total y, por tanto, contribuyen como cero a efectos de este cálculo provisional. El ratio de endeudamiento y el remanente se obtienen cuando, además, se dispone de los ingresos netos mensuales. La edad al vencimiento se calcula mediante división entera, considerando únicamente los años completos incluidos en el plazo.

Esta estrategia permite que Drools identifique determinadas condiciones de inviabilidad antes de realizar consultas externas. Por ejemplo, si los compromisos

ya conocidos superan el umbral de endeudamiento establecido, la solicitud puede evaluarse sin consultar previamente la información de CIRBE, ya que esta solo empeoraría la situación económica del cliente. Esta decisión reduce consultas que no aportarían valor a la evaluación en ese escenario, aunque implica que los indicadores calculados antes de completar las consultas externas son provisionales y pueden cambiar cuando se incorpora nueva información.

Por último, debe delimitarse el alcance del modelo financiero implementado. Las fórmulas y los datos empleados constituyen una simplificación deliberada del proceso de concesión de crédito, desarrollada en un entorno académico. La cuota estimada se obtiene mediante una división lineal del capital entre el número de meses, sin incorporar intereses, comisiones ni otras condiciones contractuales. Asimismo, conceptos como las deudas mensuales o la información relativa a avalistas se representan de forma simplificada y no abarcan toda la casuística de un entorno bancario real. La consulta de CIRBE y ASNEF se simula mediante un servicio externo independiente de la base de datos de clientes, con el propósito de representar la incorporación de información procedente de otros sistemas. Una integración bancaria real requeriría modelos de datos más detallados, criterios financieros adicionales y mecanismos de interoperabilidad y validación que quedan fuera del alcance de este trabajo. Estas simplificaciones permiten concentrar el análisis en el objetivo principal del proyecto: estudiar la interacción entre la interpretación neuronal del lenguaje natural y la evaluación simbólica mediante reglas de negocio.

## 3.5. Orquestación del flujo conversacional

La clase `PipelineOrchestrator` coordina la ejecución de cada turno conversacional mediante el método `procesar_turno_conversacional()`. Su responsabilidad principal es ejecutar las acciones requeridas por los componentes del sistema y gestionar las iteraciones necesarias hasta obtener un resultado para el turno.

### 3.5.1. Ciclo de ejecución y coordinación de acciones

En cada turno, el orquestador procesa la entrada del usuario mediante la extracción NLU y actualiza el `InformationState`. Cuando corresponde (solamente durante el primer turno de la conversación), incorpora los datos del core bancario y envía el estado disponible al motor Drools para su evaluación.

A partir del estado de la solicitud y de las reglas de negocio, Drools determina el siguiente paso del proceso, que puede implicar solicitar información adicional, consultar CIRBE/ASNEF o establecer un resultado de evaluación. El orquestador interpreta la acción indicada y ejecuta las operaciones técnicas necesarias para llevarla a cabo.

Finalmente, Mistral genera la respuesta en lenguaje natural a partir del estado resultante y de las indicaciones de la evaluación. De este modo, la interpretación lingüística y la formulación de respuestas se delegan en el componente neuronal, mientras que el control funcional del proceso permanece en Drools.

Esta separación permite distinguir entre la decisión funcional y su ejecución: el motor simbólico determina *qué debe ocurrir*, mientras que el orquestador se encarga de ejecutar técnicamente la acción correspondiente. Aunque la implementación actual contempla un conjunto limitado de acciones (únicamente la consulta de CIRBE/ASNEF), el diseño podría ampliarse incorporando nuevas operaciones y definiendo, mediante reglas en el motor simbólico, las condiciones para activarlas.

Asimismo, podría haberse incorporado una regla inicial que evaluase si la información de entrada cumplía las condiciones necesarias a nivel estructural para proceder a la evaluación simbólica y, en caso contrario, solicitase la intervención del componente neuronal para realizar la extracción NLU. Sin embargo, dado que esta extracción se realiza en cada turno conversacional antes de evaluar la solicitud, introducir una regla específica para decidir si debe ejecutarse habría añadido complejidad sin aportar una ventaja funcional significativa en el escenario implementado. Esta alternativa ilustra, no obstante, cómo el motor simbólico podría asumir un papel más amplio en la determinación de las acciones que componen el flujo de procesamiento.

### 3.5.2. Gestión del contexto conversacional

Para mantener la coherencia a lo largo de las interacciones, la arquitectura no almacena el estado en la memoria de la aplicación. En su lugar, el orquestador persiste en la base de datos el estado actualizado de la solicitud, lo que permite recuperarlo al inicio de cada nuevo turno conversacional.

FastAPI reconstruye el contexto leyendo el último `InformationState` persistido en la base de datos mediante el método `AuditTracker.obtener_ultimo_estado()`. Tras aplicar las acciones correspondientes (extracción NLU, evaluación en Drools o consulta a los *mocks*), el nuevo estado consolidado se persiste como un registro independiente. Este diseño permite que el orquestador backend opere de forma completamente *stateless*, delegando la persistencia de la sesión a MariaDB y facilitando la reconstrucción íntegra de la auditoría paso a paso.

Por otro lado, para construir el contexto enviado a Mistral en la tarea de NLU, el sistema recupera de la tabla de auditoría los últimos cuatro intercambios conversacionales, identificando las acciones `EXTRACT_NLU` y `GENERATE_NATURAL_RESPONSE`. De ellas obtiene, respectivamente, los mensajes del usuario almacenados en `step_input` y las respuestas del sistema registradas en `step_output`.

## 3.6. Componente neuronal: Mistral

Como se ha descrito en la arquitectura general, el modelo de lenguaje se utiliza para interpretar las entradas en lenguaje natural y formular las respuestas del sistema, sin asumir la responsabilidad de evaluar las reglas de negocio. La clase `MistralClient` define estas funcionalidades mediante dos métodos principales: `extraer_nlu()` y `generar_nlg()`, configurados mediante *system prompts* específicos para cada tarea.

### 3.6.1. Extracción estructurada (NLU)

La extracción de información se realiza mediante una llamada a `chat.parse()` con `temperature = 0.0`, una configuración orientada a reducir la variabilidad de las respuestas, tratando de evitar alucinaciones. La salida se estructura conforme al esquema `EsquemaNLU`, definido para restringir los tipos y los campos admitidos.

El principal elemento de control de esta fase es la ingeniería del *system prompt*, que combina instrucciones explícitas con una estrategia de *few-shot prompting*. Esta última consiste en proporcionar al modelo ejemplos de mensajes de usuario junto con sus correspondientes salidas estructuradas, ilustrando el comportamiento esperado ante distintos tipos de entrada.

Las instrucciones establecen una política de extracción conservadora: el modelo debe identificar la información aportada en el último mensaje del usuario y utilizar el historial únicamente para interpretar respuestas vinculadas al contexto inmediato, sin volver a extraer datos proporcionados en turnos anteriores. Asimismo, el *prompt* define criterios de normalización para convertir expresiones coloquiales en valores estructurados, por ejemplo, transformar una cantidad expresada como "doce mil" en `12000.0` o un plazo de tres años en 36 meses. La posterior fusión acumulativa de los datos extraídos con el `InformationState` permite conservar la información obtenida en turnos anteriores cuando el mensaje actual no aporta un nuevo valor.

Los ejemplos incluidos en el *prompt* cubren distintos escenarios, como saludos, solicitudes de préstamo, declaración de gastos y deudas, y consultas informativas. De este modo, el *few-shot prompting* ayuda a ilustrar tanto el formato de salida esperado como los criterios de extracción y clasificación aplicables.

Por ejemplo se incluye el siguiente caso, en el que el modelo debe identificar la intención de solicitar un préstamo y extraer tanto sus características como la información financiera declarada:

```
Usuario: "Necesito 8000 euros a 2 años para hacer una reforma.
Pago 650 de alquiler y tengo una deuda de 5000 euros en otro banco."

JSON: {
  "intencion_principal": "SOLICITAR_PRESTAMO",
  "cantidad_solicitada": 8000.0,
  "plazo_meses": 24,
  "proposito": "REFORMA",
  "gastos_fijos_mensuales": 650.0,
  "deuda_declarada": 5000.0,
  "duda_catch_all": "NINGUNA"
}
```

El ejemplo muestra cómo las instrucciones del *prompt* guían la normalización del plazo, la clasificación del propósito y la distinción entre gastos fijos y deudas externas, proporcionando una referencia concreta del comportamiento esperado.

### 3.6.2. Generación de respuestas (NLG)

Una vez disponible el resultado de la evaluación simbólica, el orquestador solicita a Mistral la redacción de la respuesta mediante `chat.complete()`. La configuración utiliza `temperature = 0.2` y un límite de `max_tokens = 200`, con el objetivo de favorecer respuestas breves y reducir la generación de contenido innecesario.

Para contextualizar esta generación, el modelo recibe información estructurada sobre el resultado de la evaluación. Este contexto incluye, según corresponda, el estado de la solicitud, los datos pendientes, la referencia normativa, la justificación de la decisión y el tono requerido.

El *system prompt* de esta fase establece las siguientes restricciones de comportamiento:

- **Fidelidad a la evaluación:** no debe tomar decisiones crediticias por su cuenta, inventar información ni introducir cálculos que no estén respaldados por los datos proporcionados.
- **Protección de los detalles internos:** no debe revelar al usuario nombres de variables internas, identificadores de reglas ni estructuras JSON de implementación.
- **Adecuación del tono:** debe ajustar la respuesta al valor de `tono_mensaje` indicado por el motor de reglas, por ejemplo, *"firme, claro y respetuoso"* o *"amable y orientativo"*.
- **Continuidad conversacional:** debe evitar volver a solicitar información que ya conste como disponible en el contexto proporcionado.

Estas instrucciones actúan como mecanismos preventivos para orientar la generación y mantener la coherencia entre la evaluación simbólica y la respuesta comunicada. No constituyen, por sí solas, una garantía absoluta de cumplimiento, por lo que la separación de responsabilidades entre el modelo neuronal y el motor simbólico sigue siendo un principio central del diseño.

## 3.7. Componente simbólico (Drools)

El componente simbólico se implementa mediante Drools, un motor de reglas que permite expresar las políticas de crédito de forma declarativa. La integración se realiza a través del controlador `RuleController`, que expone el *endpoint* `POST /api/evaluate`, utilizado por el cliente Python.

Por cada petición, el servicio `DroolsService` crea una nueva `KieSession`, inserta los bloques de datos del `InformationState` como hechos independientes y ejecuta las reglas mediante `fireAllRules()`. Una vez obtenida la evaluación, la sesión se cierra en un bloque `finally`. La creación de una sesión por petición permite aislar la memoria de trabajo de cada evaluación, evitando compartir los hechos entre solicitudes concurrentes.

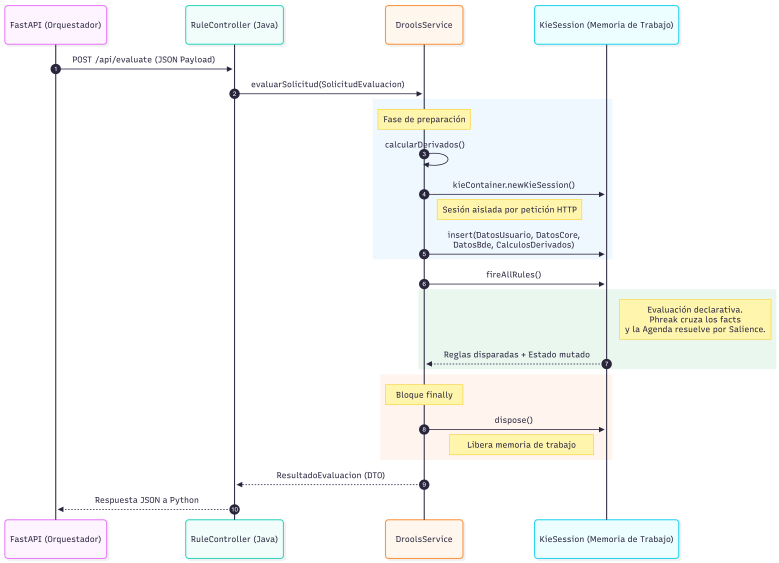



Figura 3.1: Ciclo de vida completo de una petición de evaluación a través del componente simbólico

### 3.7.1. Priorización de activaciones

Drools permite que varias reglas sean candidatas a ejecutarse cuando sus condiciones se satisfacen simultáneamente. Para controlar qué decisión prevalece, la implementación combina el atributo `salience`, que establece la prioridad relativa de las activaciones, con una condición de exclusión basada en el estado de la evaluación.

El motor utiliza el objeto `ResultadoEvaluacion`, inicializado con `estado = "EN_PROCESO"`. Todas las reglas decisorias incluyen la siguiente condición:

```
$res : ResultadoEvaluacion(estado == "EN_PROCESO")
```

Cuando una regla toma una decisión, actualiza el estado del objeto y notifica el cambio al motor mediante `update($res)`. De este modo, las reglas que requieren que el estado sea `EN_PROCESO` dejan de satisfacer esa condición una vez que el estado cambia. Este mecanismo reduce el riesgo de que varias reglas decisorias sobrescriban el resultado de una misma evaluación.

El atributo `salience` establece la prioridad relativa de las activaciones disponibles: cuanto mayor es su valor, mayor es su prioridad. No obstante, la prioridad no garantiza por sí sola la corrección del resultado, ya que este también depende de las condiciones definidas en cada regla y de las actualizaciones realizadas durante la ejecución.

Como ejemplo, la regla `R_RATIO_ENDEUDAMIENTO` deniega la solicitud cuando el ratio de endeudamiento calculado supera el 40 % de los ingresos netos mensuales:

```
rule "R_RATIO_ENDEUDAMIENTO"
    salience 92
    when
```

```
        $res : ResultadoEvaluacion(estado == "EN_PROCESO")
        $calc : CalculosDerivados(
            ratioEndeudamiento != null,
            ratioEndeudamiento > 0.40
        )
    then
        $res.setEstado("DENEGADO");
        $res.getReglasDisparadas().add("R_RATIO_ENDEUDAMIENTO");
        $res.setArticuloNormativo("Art. 18 Directiva 2014/17/UE");
        $res.setTono("firme, claro y respetuoso");
        update($res);
end
```

La regla establece una prioridad de 92 y comprueba tanto que la evaluación siga en curso como que el ratio calculado supere el umbral definido. Si ambas condiciones se cumplen, modifica el estado de la evaluación, registra la regla ejecutada y añade metadatos que permiten contextualizar la decisión y orientar su comunicación.

La siguiente tabla recoge las prioridades asignadas a las reglas del prototipo y el estado que establece cada una al ejecutarse. Se incluyen estados de denegación, solicitud de información, ejecución de acciones externas, supervisión humana y aprobación.

| **Regla** | **Salience** | **Estado resultante** |
|---|---|---|
| R_EDAD_MINIMA | 100 | DENEGADO |
| R_EDAD_MAXIMA | 95 | DENEGADO |
| R_RATIO_ENDEUDAMIENTO | 92 | DENEGADO |
| R_MINIMO_SUBSISTENCIA | 91 | DENEGADO |
| R_PROPOSITO_NO_FINANCIABLE | 90 | DENEGADO |
| R_CANTIDAD_Y_PLAZO_REQUERIDOS | 85 | REQUIERE_DATOS |
| R_PROPOSITO_REQUERIDO | 80 | REQUIERE_DATOS |
| R_GASTOS_FIJOS_REQUERIDOS | 75 | REQUIERE_DATOS |
| R_DISPARAR_CIRBE | 70 | REQUIERE_ACCION |
| R_MOROSIDAD_ASNEF | 60 | DENEGADO |
| R_ALERTA_QUIEBRA_AVAL | 50 | DENEGADO |
| R_DISCREPANCIA_DECLARADA | 40 | REQUIERE_SUPERVISION_HUMANA |
| R_APROBACION_ESTANDAR | 10 | APROBADO |

Tabla 3.2: Catálogo de reglas implementadas por *salience* descendente.

## 3.8. Implementación de las reglas de negocio

El archivo `prestamos.drl` contiene las 14 reglas que definen el comportamiento decisorio del prototipo. Las agrupamos según su función: elegibilidad y viabilidad financiera, recopilación de información, consulta de fuentes externas, gestión de discrepancias y aprobación.

### 3.8.1. Elegibilidad y viabilidad financiera

Las reglas de elegibilidad comprueban condiciones básicas de acceso al producto. `R_EDAD_MINIMA` deniega las solicitudes de usuarios menores de 18 años, mientras que `R_EDAD_MAXIMA` deniega aquellas cuyo titular superaría los 75 años al vencimiento previsto del préstamo. Por su parte, `R_PROPOSITO_NO_FINANCIABLE` excluye los propósitos clasificados como `CRIPTOMONEDAS`, `APUESTAS` o `REFINANCIACION_INFORMAL`.

La viabilidad financiera se evalúa mediante indicadores calculados a partir de la información disponible. `R_RATIO_ENDEUDAMIENTO` deniega la solicitud cuando el ratio de endeudamiento supera el 40 % de los ingresos netos mensuales. Por su parte, `R_MINIMO_SUBSISTENCIA` deniega cuando el remanente mensual calculado es inferior a 700 euros.

Los umbrales de edad máxima, endeudamiento y remanente corresponden a políticas de diseño del prototipo, definidas en las reglas DRL, y no deben interpretarse como límites legales universales.

### 3.8.2. Recopilación de información y consulta externa

Las reglas de información pendiente comprueban si están disponibles los datos necesarios para continuar la evaluación. `R_CANTIDAD_Y_PLAZO_REQUERIDOS` identifica la ausencia de la cantidad solicitada o del plazo. `R_PROPOSITO_REQUERIDO` solicita el propósito cuando los datos anteriores están disponibles. `R_GASTOS_FIJOS_REQUERIDOS` señala la ausencia de los gastos fijos mensuales, tales como los gastos por el alquiler de la vivienda, suministros, seguros, etc. Estas reglas producen el estado `REQUIERE_DATOS` e identifican mediante `dato_faltante` la información que debe completarse, permitiendo una recopilación progresiva de los datos.

Cuando se dispone de la información necesaria y todavía no se ha realizado la consulta externa, `R_DISPARAR_CIRBE` establece el estado `REQUIERE_ACCION` y asigna a `accion_requerida` el valor `CONSULTAR_CIRBE_ASNEF`. El orquestador ejecuta entonces la operación correspondiente contra el servicio simulado e incorpora los resultados al estado de la solicitud antes de continuar con la evaluación.

### 3.8.3. Riesgo externo, discrepancias y supervisión humana

Las reglas basadas en información externa permiten evaluar circunstancias que no pueden determinarse únicamente a partir de los datos proporcionados por el usuario. `R_MOROSIDAD_ASNEF` deniega la solicitud cuando, tras realizarse la consulta, se detectan impagos activos o un valor de morosidad ASNEF superior a cero. `R_ALERTA_QUIEBRA_AVAL` deniega cuando el usuario figura como avalista de una empresa en concurso de acreedores.

La regla `R_DISCREPANCIA_DECLARADA` detecta diferencias entre la deuda declarada por el usuario y la deuda total registrada en CIRBE. La condición implementada comprueba si la deuda registrada supera en más de un 35 % la declarada:

```
eval($bde.getDeudaCirbeTotal() >
     ($du.getDeudaDeclarada() * 1.35))
```

Cuando se cumple esta condición, el motor establece el estado `REQUIERE_SUPERVISION_HUMANA` en lugar de denegar automáticamente la solicitud. Esta decisión de diseño permite derivar a revisión las discrepancias que requieren comprobación adicional, en lugar de tratarlas directamente como evidencia concluyente de riesgo.

### 3.8.4. Aprobación y justificación de las decisiones

`R_APROBACION_ESTANDAR` es la regla que produce el estado `APROBADO`. Entre sus condiciones se encuentran un ratio de endeudamiento no superior al 40 %, un remanente mensual de al menos 700 euros, la consulta externa realizada, la ausencia de impagos activos y una morosidad ASNEF igual a cero. La aprobación representa, por tanto, el cumplimiento de los criterios codificados en el prototipo.

Las reglas incorporan metadatos que permiten contextualizar la decisión del motor simbólico:

- `articuloNormativo`: almacena la referencia normativa o el criterio regulatorio asociado a la decisión. Su contenido sirve como contexto para explicar el fundamento de la regla aplicada, aunque no siempre corresponde a un artículo concreto de una disposición legal.
- `textoLegal`: contiene la justificación asociada a la decisión, preparada durante la definición de la regla. Explica el motivo de la decisión o la información que se necesita para continuar la evaluación y proporciona al modelo neuronal una base de contenido para formular la respuesta.
- `tono`: especifica las características comunicativas esperadas de la respuesta, mediante indicaciones como *amable, orientativo y conciso* o *claro, respetuoso y transparente*. Su finalidad es orientar la forma de expresar la decisión, sin modificar el estado establecido por el motor simbólico ni los criterios que han determinado el resultado.

Estos elementos se proporcionan a Mistral para que formule la explicación dirigida al usuario a partir de información determinada por el motor de reglas, evitando que el modelo genere por su cuenta los fundamentos de la decisión.

### 3.8.5. Verificación del motor de reglas

La verificación del comportamiento del motor simbólico se articula mediante dos baterías de pruebas complementarias ubicadas en el directorio `tests/`:

- **Verificación unitaria del catálogo (`test_drools_rules.py`):** Esta batería comprueba la correcta implementación y activación individual de cada regla de negocio. La estrategia parte de un perfil base que cumple todos

los criterios de concesión estándar (`R_APROBACION_ESTANDAR`) y altera de forma aislada los datos pertinentes para evaluar la respuesta ante cada condición específica. De este modo, se comprueba que cada regla se dispare ante sus premisas específicas y asigne el estado, fundamentación jurídica y tono requeridos.

- **Verificación de prioridades y resolución de conflictos (`test_drools_salience.py`):** Dado que en un escenario real una solicitud puede activar múltiples reglas a la vez, esta batería valida el comportamiento del motor ante condiciones concurrentes. Las pruebas configuran casos en los que se satisfacen simultáneamente las premisas de dos o más reglas en distintos escalones de prioridad (por ejemplo, minoría de edad frente a ratio financiero, propósito no financiable frente a consulta de CIRBE, o morosidad en ASNEF frente a discrepancia declarada). La batería verifica que la regla de mayor *salience* resuelva el conflicto y que la guarda (`estado == "EN_PROCESO"`), unida a la actualización del hecho (`update($res)`), inhiba la ejecución de las reglas de menor prioridad, garantizando que no se sobreescriba la decisión.

La combinación de ambas baterías permite certificar tanto la validez de las reglas aisladas como el determinismo y la robustez del motor ante conflictos normativos.

## 3.9. Entorno bancario simulado

El prototipo utiliza un entorno bancario simulado que permite representar tanto la información interna de los clientes como las consultas a fuentes externas de solvencia. Este entorno se compone de una base de datos que representa el *core* bancario y un servicio que simula las consultas a CIRBE y ASNEF.

### 3.9.1. Simulación del *core*

La información interna se almacena en la tabla relacional `clientes`, cuyo esquema se define en `database/init.sql`. Esta tabla contiene los datos identificativos y financieros necesarios para la evaluación, entre ellos `cliente_id`, `dni`, `edad`, `ingresos_netos_mensuales` y `antiguedad_meses`, así como los campos `es_avalista` y `empresa_avalada_cif`, relacionados con posibles avales. El campo `deuda_interna_banco_mensual` representa las cuotas mensuales de préstamos activos con la propia entidad y se incorpora al cálculo de los compromisos financieros del solicitante.

La base de datos se inicializa con cuatro perfiles de prueba que permiten comprobar distintos escenarios del motor de reglas:

- **CLI-001**: perfil sin incidencias, utilizado para comprobar el flujo de aprobación estándar.

- **CLI-002**: cliente que actúa como avalista de una empresa en concurso de acreedores.
- **CLI-003**: perfil con morosidad activa registrada en fuentes externas.
- **CLI-004**: solicitante próximo al límite de edad máxima establecido en el prototipo.

### 3.9.2. Simulación de consultas externas de solvencia

Las consultas a CIRBE y ASNEF se simulan mediante la clase `BdeMockService`, que utiliza el archivo `bde.json` para recuperar la información asociada a cada DNI. Los datos incluyen la existencia de impagos activos, el importe de la morosidad en ASNEF, la situación concursal de la empresa avalada y los importes de deuda mensual y total registrados en CIRBE. El servicio también contempla una entrada `_default`, sin incidencias, que se utiliza cuando el DNI consultado no está incluido explícitamente en el archivo.

Además, las operaciones de consulta simuladas se exponen mediante *endpoints* REST que pueden inspeccionarse y ejecutarse desde la documentación interactiva de Swagger, disponible en la ruta `/docs` de FastAPI. Esto permite comprobar las respuestas del servicio para distintos DNI sin tener que recorrer todo el flujo conversacional.

## 3.10. Persistencia y trazabilidad

El sistema utiliza MariaDB como capa de persistencia relacional para registrar las interacciones y los pasos de procesamiento de cada evaluación. Este registro permite relacionar las entradas, salidas y estados a las operaciones del sistema, facilitando el seguimiento y el análisis retrospectivo de las decisiones.

### 3.10.1. Esquema de la tabla `trazas_auditoria`

La tabla `trazas_auditoria` centraliza los registros de auditoría de la aplicación. Su estructura permite identificar cada operación, situarla dentro de una sesión y un turno conversacional y conservar información sobre su ejecución. Las principales columnas son:

- **`id`**: Clave primaria autoincremental de tipo `BIGINT`, que identifica de forma única cada registro y permite establecer un orden de referencia entre ellos.
- **`session_id`**: Identificador de la sesión conversacional, almacenado como `VARCHAR(64)`. Permite agrupar los registros correspondientes a una misma sesión.
- **`turn_id`**: Número de turno (`INT`), que identifica la interacción dentro de la sesión.

- **step_order**: Orden del paso dentro de un turno (INT), que permite reconstruir la secuencia de operaciones registradas.
- **action_type**: Tipo de operación ejecutada (VARCHAR(64)), como EXTRACT_NLU o EVALUATE_DROOLS.
- **step_input**: Campo de tipo JSON que almacena los datos de entrada registrados para la operación.
- **step_output**: Campo de tipo JSON que conserva la salida registrada de la operación, antes de las transformaciones posteriores que puedan producirse en el flujo.
- **information_state**: Campo de tipo JSON que almacena una copia del estado de la solicitud asociado al paso registrado, lo que permite consultar qué información se encontraba disponible en ese punto del procesamiento.
- **created_at**: Marca temporal de tipo TIMESTAMP, generada por la base de datos, que indica cuándo se creó el registro.

Además, para agilizar las consultas de auditoría, la tabla dispone de índices sobre session_id, la combinación de session_id y turn_id, action_type y created_at.

### 3.10.2. Registro de las operaciones

La escritura de los registros corresponde a la clase AuditTracker, ubicada en services/audit_tracker.py. Desde la aplicación, el registro sigue un enfoque *append-only*: los registros históricos se conservan sin actualizaciones ni eliminaciones durante el funcionamiento ordinario del sistema.

La persistencia se realiza de forma síncrona después de cada paso del orquestador. La utilización de db.add() y db.commit() permite guardar y confirmar cada registro antes de continuar con la siguiente operación. De este modo, cada paso dispone de su propio registro confirmado.

### 3.10.3. Reconstrucción de la evaluación

Los registros permiten analizar retrospectivamente una evaluación a partir de la información conservada en cada paso. En particular, pueden utilizarse para examinar los siguientes aspectos:

1. **Información disponible:** el campo information_state permite consultar el estado de la solicitud en un determinado instante.
2. **Entrada y salida de la evaluación:** los campos step_input y step_output permiten inspeccionar, respectivamente, los datos registrados como entrada y el resultado devuelto por la operación.

3. **Secuencia de operaciones:** la combinación de `session_id`, `turn_id`, `step_order` y `action_type` permite reconstruir la secuencia de pasos registrados durante el procesamiento de la solicitud.

4. **Respuesta dirigida al usuario:** si la salida de `GENERATE_NATURAL_RESPONSE` conserva el texto generado, es posible recuperar la respuesta registrada y compararla con la decisión y las instrucciones proporcionadas por el componente simbólico.

En conjunto, este mecanismo proporciona trazabilidad sobre las operaciones registradas y facilita la revisión de las decisiones del prototipo.

## 3.11. Interfaz de usuario y consola de auditoría

La interfaz web del prototipo se ha desarrollado con el objetivo de permitir la interacción conversacional con el sistema y facilitar la inspección de su funcionamiento interno. Su finalidad principal es proporcionar un entorno de demostración en el que puedan observarse tanto el desarrollo de la conversación como el procesamiento de la solicitud por los componentes neuronal y simbólico.

Para ello, la aplicación combina un panel conversacional con una consola que muestra información sobre las fases de procesamiento, los resultados de la evaluación de las reglas y el estado de la solicitud. Esta disposición permite relacionar las respuestas visibles en la conversación con los datos y resultados que intervienen en su generación, facilitando el análisis experimental de la trazabilidad del prototipo.

### 3.11.1. Diseño de la interfaz y distribución de los paneles

La interfaz utiliza una disposición de pantalla dividida, con dos paneles complementarios y un divisor redimensionable:

- **Panel izquierdo: interacción conversacional.** Permite simular la conversación de una solicitud de crédito con el perfil de un cliente. Incluye un selector de clientes de prueba que permite cargar perfiles previamente definidos en el repositorio de datos del prototipo. Estos elementos facilitan la reproducción de distintos escenarios durante las pruebas, pero no implementan un proceso real de autenticación bancaria. Además, con la selección de cada cliente, se muestran en el frontal su id, nombre completo y sus ingresos y edad.

- **Panel derecho: consola de inspección y auditoría.** Presenta información técnica sobre el procesamiento de la solicitud, incluido el estado del flujo de ejecución, los resultados de la evaluación simbólica y los datos estructurados que componen el estado de la solicitud (*Information State*). Su finalidad es facilitar la observación del comportamiento del sistema durante las interacciones y relacionar los resultados mostrados con las operaciones realizadas.

El divisor situado entre ambos paneles permite ajustar la anchura relativa de las áreas de conversación e inspección. De este modo, la distribución puede adaptarse a las necesidades de cada prueba, dando mayor espacio al diálogo o a la información técnica, según corresponda.

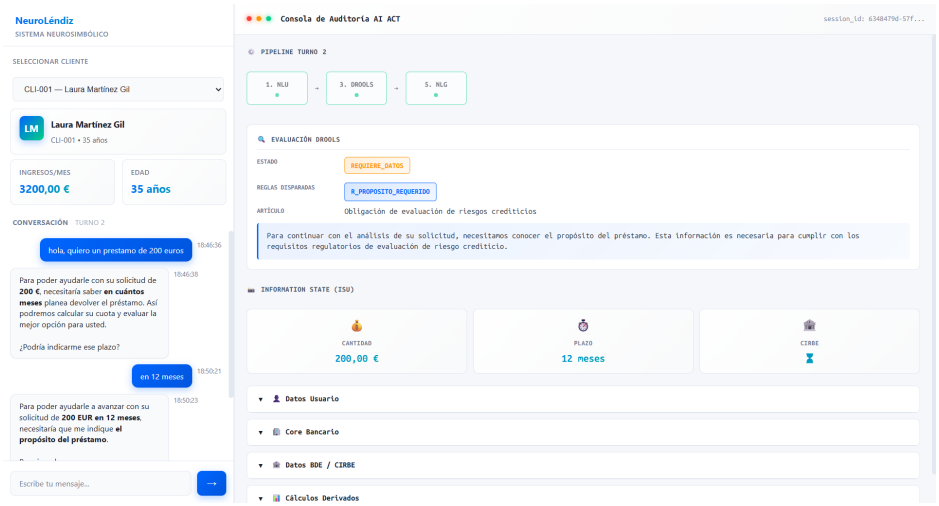



Figura 3.2: Vista general de la interfaz experimental, con el panel conversacional y la consola de inspección del sistema.

### 3.11.2. Implementación técnica y comunicación con el backend

La capa de presentación, por simplicidad, se ha implementado mediante tecnologías web nativas, sin recurrir a tecnologías de frontend como React o Angular. La solución utiliza HTML para definir la estructura de la página, CSS para su presentación visual y JavaScript para gestionar las interacciones y las comunicaciones con el backend. Esta elección mantiene sencilla la implementación del prototipo y evita introducir dependencias adicionales para las funcionalidades requeridas.

Los archivos estáticos se encuentran en el directorio `backend/app/static/` y se sirven desde FastAPI a través de la ruta `/static`. La implementación se distribuye en los siguientes archivos:

- `index.html`: define la estructura de la interfaz y los elementos que componen los paneles conversacional y técnico.
- `styles.css`: contiene los estilos visuales, las variables CSS, la distribución de los paneles y los estados visuales de los componentes. Incluye también los estilos asociados a la presentación de los datos y a los elementos interactivos.
- `app.js`: gestiona las interacciones del navegador, el redimensionamiento de los paneles, el comportamiento de los acordeones y las peticiones asíncronas a la API REST.

La comunicación con el backend se realiza mediante peticiones HTTP. Al iniciar la interfaz, el archivo `app.js` consulta el endpoint `/api/clientes`, expuesto a través de `clientes_router.py`, para obtener los perfiles de prueba disponibles. Cuando se selecciona un cliente o se envía un mensaje, el frontend realiza una petición `POST` a `/api/chat`, que incluye el identificador del cliente, el mensaje y el identificador de sesión (`session_id`).

La respuesta del backend contiene la información que necesita la interfaz para actualizar la conversación y los elementos de inspección. A partir de ella, el frontend incorpora la respuesta textual del asistente al historial y actualiza los componentes que representan el estado de la solicitud y los resultados del procesamiento.

### 3.11.3. Consola de inspección

La consola técnica permite observar distintos aspectos del procesamiento de cada interacción. Su función es hacer visible la información relevante para analizar el comportamiento del prototipo y distinguir las responsabilidades de los componentes que intervienen en la conversación.

Entre los elementos principales se encuentran los siguientes:

1. **Identificador de la sesión.** En la cabecera superior se expone el identificador único de la sesión (`session_id`). Esta clave es fundamental, ya que es el nexo que permite relacionar la interacción que se está visualizando en la interfaz con los registros almacenados en la base de datos MariaDB.

2. **Monitorización del Pipeline del Turno.** Inmediatamente debajo, el sistema representa visualmente la secuencia de micro-acciones ejecutadas por el orquestador para el turno conversacional en curso. Este diagrama dinámico demuestra que la orquestación es modular y que la ejecución sigue los pasos discretos del paradigma *Symbolic[Neuro]*, reflejando únicamente las acciones que se han invocado en esa iteración.

3. **Resultados de la Evaluación Drools.** Este bloque expone el veredicto del motor de reglas de negocio. Incluye:

   - *Estado:* El dictamen final de la iteración.
   - *Reglas disparadas:* El identificador exacto de la regla DRL que ha forzado dicho estado.
   - *Justificación normativa:* Desglosada en la referencia legal aplicable y el texto explicativo asociado. Al mostrarse en este bloque, se evidencia que la justificación legal es un metadato inyectado por Drools, y no una alucinación generada por el LLM.

La consola permite, por tanto, examinar el resultado de Drools junto con parte de la información utilizada para contextualizarlo.




Figura 3.3: Detalle de la consola de inspección: fases del procesamiento y resultado de la evaluación mediante Drools.

Además, la parte inferior de la consola presenta el estado estructurado de la solicitud (*Information State*). Su visualización hace transparente el contexto del sistema, permitiendo examinar qué información está disponible y cómo evoluciona a medida que avanza la interacción, sin necesidad de consultar el registro de la base de datos.

En la zona superior de esta sección se muestran indicadores resumidos que ofrecen una vista rápida de los datos más importantes de la operación, como el importe solicitado, el plazo y el estado de la consulta a CIRBE. A continuación, cuatro paneles desplegables permiten inspeccionar la información agrupada por su procedencia: datos declarados por el usuario, información del *core* bancario, datos de fuentes externas y cálculos financieros derivados.

Esta agrupación visual facilita la distinción entre la información aportada por el usuario y los valores validados o calculados por el sistema.

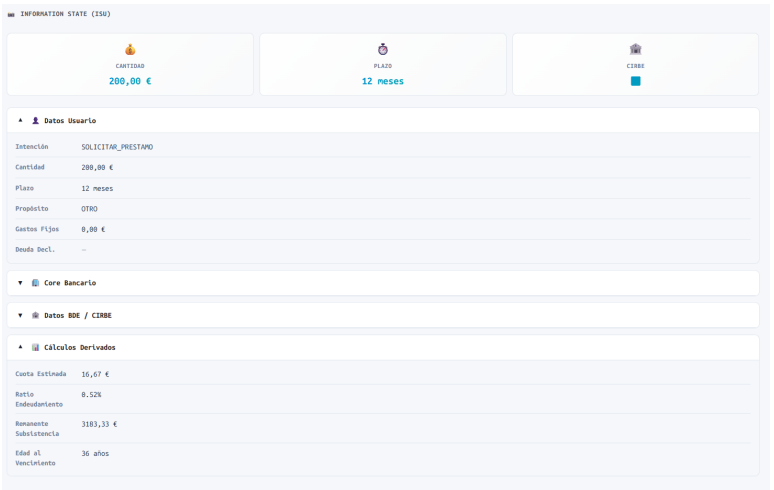



Figura 3.4: Detalle del estado estructurado de la solicitud: indicadores resumidos y datos agrupados por procedencia.

En conjunto, la interfaz constituye una herramienta de apoyo a la demostración y evaluación del prototipo neuro-simbólico. Al combinar la interacción conversa-

cional con la visualización de los resultados simbólicos y del estado estructurado de la solicitud, facilita el análisis de la relación entre la información procesada, las reglas aplicadas y las respuestas generadas. Su alcance es experimental y se centra en hacer observable el funcionamiento interno del sistema, sin pretender sustituir a una interfaz de producción ni a los mecanismos formales de auditoría de un despliegue real.

## 3.12. Flujo de ejecución: ciclo de vida de un turno

Para ilustrar cómo interactúan los componentes de la arquitectura, se describe a continuación el procesamiento de un turno conversacional que requiere una consulta a fuentes externas de solvencia. El flujo comienza cuando el mensaje del usuario llega al endpoint `POST /api/chat` y continúa mediante las siguientes etapas:

1. **Recuperación del estado y comprensión del mensaje (NLU).** El backend recupera el estado de la solicitud mediante `obtener_ultimo_estado()`. Si se trata de una sesión nueva, inicializa un objeto `InformationState`; en caso contrario, recupera el estado previamente almacenado en MariaDB. A continuación, envía el mensaje actual y el contexto conversacional pertinente a Mistral, que devuelve los campos estructurados conforme al esquema `EsquemaNLU`. El backend incorpora la información extraída a `datos_usuario`, de acuerdo con las reglas de actualización del estado. Esta operación queda registrada mediante una traza de tipo `EXTRACT_NLU`, que recoge la entrada proporcionada al modelo y la salida estructurada obtenida.

2. **Recuperación de datos internos del cliente.** Cuando corresponde cargar los datos del cliente, el backend consulta la tabla `clientes` y actualiza `datos_core_bancario` con la información obtenida. Esta operación queda registrada mediante una traza `FETCH_CORE_DATA`, que recoge el identificador del cliente consultado, así como los datos devueltos por el repositorio. De este modo, es posible consultar qué información del entorno bancario simulado se incorporó al estado de la solicitud.

3. **Evaluación simbólica.** El backend construye un objeto `DroolsRequest` con el estado disponible y lo envía al motor de reglas para su evaluación. Esta operación queda registrada mediante una traza `EVALUATE_DROOLS`, que recoge los datos utilizados y el resultado devuelto por el motor, incluidas las reglas registradas, el estado de la solicitud y las acciones requeridas. Si Drools determina que es necesario consultar las fuentes externas de solvencia, devuelve el estado `REQUIERE_ACCION` y la operación `CONSULTAR_CIRBE_ASNEF`, que indica al backend qué acción debe ejecutar.

4. **Consulta externa y reevaluación.** Cuando la evaluación requiere información adicional de solvencia, FastAPI ejecuta la consulta a través de `BdeMockService`. La operación queda registrada mediante una traza

`FETCH_CIRBE_ASNEF`, que recoge los parámetros enviados al servicio y los datos obtenidos. A continuación, el backend actualiza `datos_externos_bde` con la información recibida y vuelve a enviar el estado actualizado a Drools para una nueva evaluación. Esta segunda evaluación genera otra traza `EVALUATE_DROOLS`, que permite examinar el resultado obtenido tras incorporar la información externa.

5. **Generación de la respuesta y registro de auditoría.** Una vez obtenido el resultado de la evaluación, el backend proporciona a Mistral la información necesaria para generar una respuesta en lenguaje natural. El modelo verbaliza el resultado y la explicación asociada a la evaluación, sin asumir la responsabilidad de decidir la concesión del préstamo. Esta operación queda registrada mediante una traza `GENERATE_NATURAL_RESPONSE`, que almacena la información proporcionada al modelo y la respuesta generada.

La Figura **??** representa la secuencia de interacción entre los componentes durante este escenario.

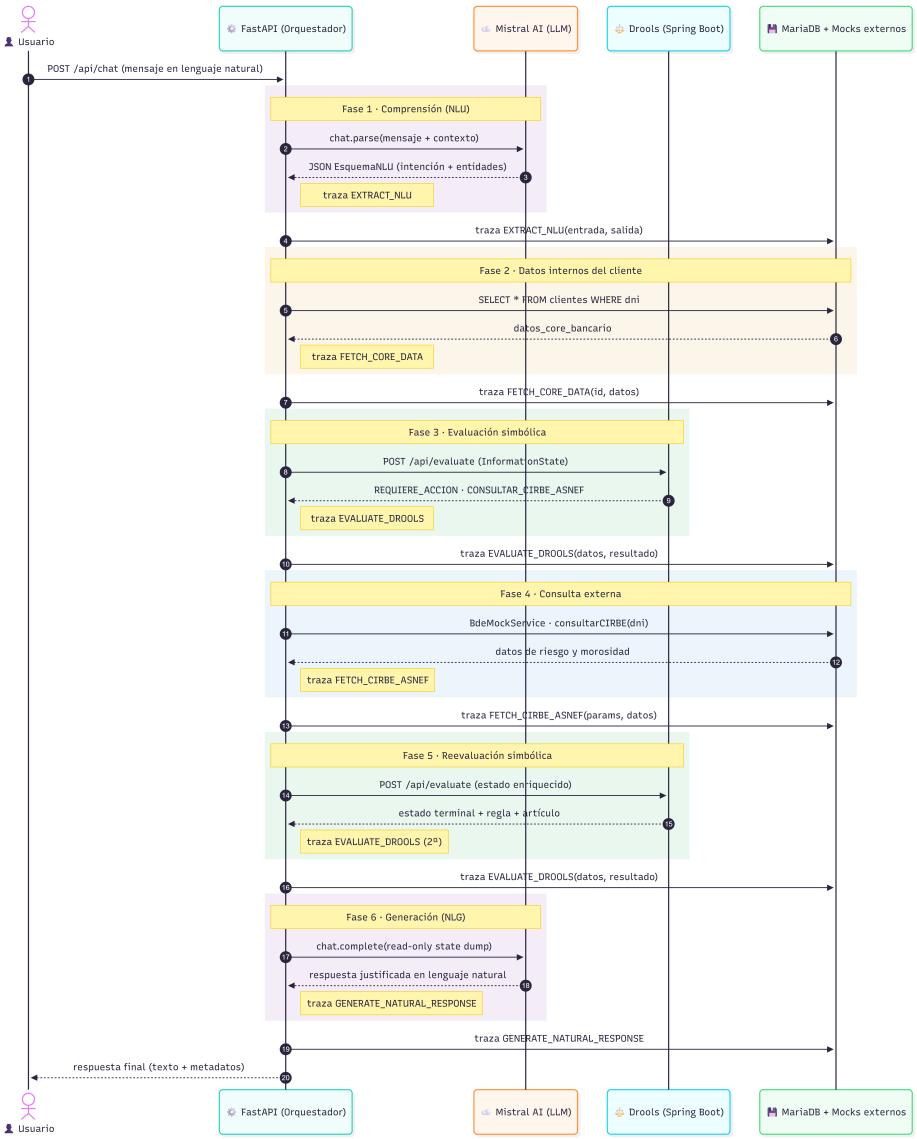



Figura 3.5: Diagrama de secuencia del procesamiento de un turno conversacional que requiere una consulta externa de solvencia.

En conjunto, la arquitectura materializa el enfoque neuro-simbólico planteado en este trabajo mediante la separación entre comprensión lingüística, evaluación basada en reglas, ejecución técnica y persistencia. Esta organización permite mantener un estado estructurado independiente del historial textual de la conversación, centralizar los criterios de evaluación crediticia en el motor simbólico y registrar las operaciones necesarias para analizar el comportamiento del prototipo.